

虚拟问诊系统设计与实现

摘要：传统医学问诊实训长期面临真实病例稀缺、标准化病人成本高、教师批量指导效率低、训练效果难以量化的困境。大语言模型的发展为模拟标准化病人提供了新的可能，但简单依赖提示词工程易出现角色混乱、临床信息编造、教学反馈缺失等问题，难以直接满足医学教育的严谨性与可控性要求。如何在通用大模型之上构建兼具沉浸交互、知识可控与评估闭环的虚拟问诊系统，已成为医学教育数字化转型中亟待解决的工程问题。

针对上述问题，本文设计并实现了一套面向医学院校的虚拟问诊系统。系统采用前后端分离架构：前端基于 Vue 3、TypeScript 与 Element Plus 构建，后端基于 Spring Boot 3.3.6、Java 17 与 MyBatis-Plus 实现，使用 MySQL 持久化业务数据，Redis 缓存会话上下文，MinIO 存储医学影像，并通过 Docker Compose 完成容器化部署。系统面向学生、教师、管理员三类角色，提供 AI 问诊训练、病例库管理、学习分析与系统管控四大功能模块，集成基于 RBAC 的细粒度权限模型、JWT 认证、ECharts 数据可视化与 SSE 流式响应等关键能力。

为突破单一提示词调用大模型的局限，本系统在通用大模型之上构建了 Agent 编排层：通过 AgentRouter 在 PatientAgent、DiagnosisAgent 和 FeedbackAgent 之间进行职责路由，以避免问诊中泄题与角色混淆；以关键词检索的轻量级 RAG 向提示词注入医学知识，规避向量库的运维成本；以基于 Redis 滑动窗口的会话记忆维护多轮上下文；以临床工具自动注入机制从病例数据返回客观检查结果，规避大模型对检查数值的幻觉；以结构化 JSON 多维评分配合规则兜底，保障评分服务的鲁棒性与可视化分析能力。系统已部署于云端服务器，并通过功能、性能与安全测试，验证了所提方案在医学问诊实训场景中的可行性与有效性。

关键词：医学教育；虚拟问诊；大语言模型；AI Agent；检索增强生成；前后端分离

Design and Implementation of a Virtual Consultation System

Abstract: Traditional medical inquiry training has long suffered from a scarcity of authentic clinical cases, the high cost of standardized patients, the low efficiency of teacher supervision, and the difficulty of quantifying training outcomes. The development of large language models (LLMs) makes it possible to simulate standardized patients, yet naive prompt engineering may cause role confusion, fabrication of clinical findings and insufficient pedagogical feedback. Therefore, building a controllable virtual consultation system with immersive interaction and closed-loop assessment has become an important task in the digital transformation of medical education.

To address these issues, this thesis designs and implements a virtual consultation system for medical schools. The system adopts a decoupled front-end / back-end architecture: the front-end is built upon Vue 3, TypeScript and Element Plus, while the back-end is built upon Spring Boot 3.3.6, Java 17 and MyBatis-Plus. MySQL, Redis and MinIO are used for business data, dialogue context and medical images, and Docker Compose is used for containerized deployment. Targeting students, teachers and administrators, the system provides AI consultation training, case-library management, learning analytics and system administration, integrated with RBAC, JWT authentication, ECharts visualization and SSE streaming responses.

To overcome the limitations of plain prompt-based LLM usage, an agent-orchestration layer is constructed atop the LLM. AgentRouter dispatches messages among PatientAgent, DiagnosisAgent and FeedbackAgent to prevent answer leakage and role confusion. A lightweight keyword-based RAG mechanism injects medical knowledge into prompts, Redis memory maintains multi-turn context, and clinical-tool auto-injection returns objective examination findings from case data to reduce hallucination. A structured JSON scoring scheme with rule-based fallback improves the robustness and analytical value of the evaluation service. The system has been deployed on a cloud server and validated through functional, performance and security tests, demonstrating its feasibility in medical inquiry training scenarios.

Keywords: Medical Education; Virtual Consultation; Large Language Model; AI Agent; Retrieval-Augmented Generation; Front-end / Back-end Separation

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 医学教育数字化研究现状	2
1.2.2 大语言模型在医学教育中的应用	2
1.2.3 AI Agent 与多智能体协作研究现状	3
1.3 研究目标与主要工作	3
1.4 论文组织结构	4
第 2 章 相关技术概述	5
2.1 前端开发技术	5
2.1.1 Vue 3 与 Composition API	5
2.1.2 Element Plus 与 ECharts	5
2.2 后端开发技术	6
2.2.1 Spring Boot 与 Spring Security	6
2.2.2 MyBatis-Plus 与 JWT	6
2.3 大语言模型与阿里百炼平台	6
2.4 AI Agent 与多智能体协作	7
2.5 检索增强生成 RAG	7
2.6 工具调用 Tool Calling / Function Calling	8
2.7 数据库与缓存技术	8
2.7.1 MySQL 8.0	8
2.7.2 Redis 7	9
2.7.3 MinIO 对象存储	9
2.8 容器化部署技术	9
2.9 本章小结	10
第 3 章 系统需求分析	11
3.1 业务需求分析	11
3.1.1 医学问诊实训现状与痛点	11
3.1.2 用户角色分析	11

3.2 功能需求分析	12
3.2.1 学生端功能需求	12
3.2.2 教师端功能需求	12
3.2.3 管理员端功能需求	13
3.2.4 公共与核心技术需求	13
3.3 非功能需求分析	14
3.3.1 性能需求	14
3.3.2 安全需求	14
3.3.3 可用性与可维护性需求	15
3.4 用例分析	15
3.5 本章小结	17
第 4 章 系统总体设计	18
4.1 系统架构设计	18
4.1.1 整体架构	18
4.1.2 前后端分离架构	18
4.1.3 多 Agent 协同架构	19
4.2 功能模块划分	20
4.3 数据库设计	20
4.3.1 ER 图	20
4.3.2 主要数据表设计	21
4.3.3 Redis 数据结构	22
4.4 接口设计	22
4.5 RBAC 权限模型设计	24
4.6 安全架构设计	25
4.7 本章小结	25
第 5 章 AI Agent 核心技术实现	26
5.1 多 Agent 路由架构	26
5.1.1 Agent 抽象与配置	26
5.1.2 PatientAgent: 虚拟标准化病人	27
5.1.3 DiagnosisAgent: 诊断评估	27
5.1.4 FeedbackAgent: 治疗反馈	28
5.1.5 AgentRouter 路由策略	28

5.2 RAG 知识检索增强	29
5.2.1 关键词提取与召回策略	29
5.2.2 知识上下文注入	29
5.3 会话记忆 Memory 模块	30
5.3.1 基于 Redis 的滑动窗口	30
5.3.2 为何不用 LangChain Memory	30
5.4 临床工具自动注入机制	31
5.4.1 影像类型识别与匹配	31
5.5 Function Calling 工具清单	32
5.6 结构化 AI 评分	32
5.6.1 多维度评分提示词	32
5.6.2 JSON 解析与规则兜底	33
5.7 阿里百炼大模型集成	34
5.7.1 OpenAI 兼容 API 调用	34
5.7.2 流式输出 SSE 实现	34
5.8 完整对话编排流程	34
5.9 本章小结	35
第 6 章 系统功能模块详细实现	36
6.1 学生端核心模块	36
6.1.1 AI 问诊训练界面	36
6.1.2 病例选择与训练启动	37
6.1.3 学习分析与雷达图	37
6.2 教师端核心模块	38
6.2.1 病例管理与多媒体上传	38
6.2.2 批改工作台	38
6.2.3 教学分析	39
6.3 管理员端核心模块	39
6.3.1 用户与角色管理	39
6.3.2 系统监控	40
6.3.3 日志与存储管理	40
6.4 公共模块	40
6.4.1 认证授权	40
6.4.2 文件上传	40
6.4.3 数据可视化	41

6.5 本章小结	41
第 7 章 系统测试与部署	42
7.1 测试环境	42
7.2 功能测试	42
7.3 性能测试	43
7.4 AI Agent 效果验证	44
7.5 安全测试	44
7.6 容器化部署	45
7.6.1 Docker Compose 编排	45
7.6.2 部署到服务器	46
7.6.3 部署效果	46
7.7 本章小结	47
第 8 章 总结与展望	48
8.1 研究工作总结	48
8.2 创新点	48
8.3 工作不足	49
致谢	50
参考文献	51
附录	53

第 1 章 绪论

1.1 研究背景与意义

医学问诊（Medical Inquiry）是连接医患沟通、临床思维与诊疗决策的基础环节，也是医学院校临床技能训练的核心内容之一。国家层面关于医学教育创新发展与教育信息化建设的政策均强调，要推动信息技术与人才培养深度融合，提升医学人才培养质量^[1-2]。传统问诊实训通常依赖课堂讲授、临床见习以及标准化病人（Standardized Patient, SP）演练等方式开展^[3]，长期以来面临三类未被有效解决的痛点：其一，真实病例资源稀缺，难以系统性覆盖各专科的常见病、多发病乃至疑难病例，受限于隐私与伦理约束，规模化使用真实病例的难度更高；其二，标准化病人的招募、培训与维护成本居高不下，单位时间内能够提供给学生的实训机会十分有限；其三，教师对每名学生的问诊过程进行细致复盘和点评所需投入的人力成本极大，量化、可复盘、可沉淀的教学反馈难以形成体系。在医学院校规模扩张与临床带教资源相对紧张的双重压力下，上述问题愈发突出，已成为医学教育数字化转型中亟待补齐的关键短板。

近年来，以 GPT-4、通义千问等为代表的大语言模型（Large Language Model, LLM）在自然语言理解、专业知识问答、临床推理等任务上展现出令人瞩目的能力。国内学者已经开始系统讨论 ChatGPT 对医学教育模式的影响^[4]，中文医疗大模型综述也指出，面向中文医疗语境构建专用数据集、评测体系和模型能力，是医疗大模型落地应用的重要基础^[5]。在模型实践方面，HuatuogPT^[6]、Qwen^[7]等面向中文或中文友好的大模型相继出现，进一步降低了医学场景下大模型应用的研发门槛。国外研究中，Singhal 等^[8]和 Kung 等^[9]也分别验证了大语言模型在医学问答与医学考试任务中的潜力。上述工作共同表明，将大语言模型用于模拟标准化病人、构建可 7×24 小时使用、低成本、可复盘的虚拟问诊环境已经具备技术可行性。

然而，将通用大模型直接应用于医学问诊实训仍然存在明显鸿沟。一方面，单纯的提示词调用方式难以稳定地约束模型扮演患者角色，容易出现“AI 既扮演病人又主动给出诊断”的角色混淆，破坏教学过程的严谨性；另一方面，模型在面对客观检查结果（如血常规数值、影像描述）时容易产生幻觉，违背医学教育对客观性与可控性的基本要求；此外，问诊结束后缺乏面向能力维度的结构化评估，使得训练效果难以量化沉淀，教师无法获得可视化的学情数据。针对这些问题，本课题在阿里通义千问大模型之上，引入多 Agent 协同、检索增强生成、临床工具自动注入与结构化评分等机制，设计并实现了一套面向医学院校的虚拟问诊系统。系统不仅可以缓解传统实训的资源约束，还能教师提供学情可视化、知识薄弱点分析与批改工作台等管理工具，对推动医学教育数字化与智能化升级具有重要的现实意义和应用价值。

1.2 国内外研究现状

1.2.1 医学教育数字化研究现状

医学教育数字化的探索可以追溯到 20 世纪 90 年代以计算机辅助教学为代表的电子病例库与多媒体教学系统。进入 21 世纪以来，随着 Web 技术与多媒体内容生产能力的发展，国外医学院校逐步建立了以临床综合考核（Objective Structured Clinical Examination, OSCE）数字化、虚拟病人（Virtual Patient）与高仿真模拟人（high-fidelity simulator）为代表的实训体系，部分研究还将虚拟病人与游戏化交互机制相结合，以增强学生的沉浸感和决策训练效果。在国内，随着“互联网 + 教育”政策的推进，人民卫生出版社相继建设了医学慕课和电子题库，多家医学院校依托国家级虚拟仿真实验教学项目建设了系统化的虚拟仿真实训平台，覆盖问诊、查体、急救等典型临床场景^[10]。

总体来看，传统数字化方案的核心方法是依据预设脚本组织的“决策树式问答”或“知识点填空”。其优点是流程可控、内容安全，但局限同样明显：交互的灵活性有限、难以复现真实问诊中开放式的语言表达、缺乏面向学生薄弱点的自适应反馈，整体上仍处于“内容数字化”而非“过程智能化”的阶段。这一现状构成了在医学问诊实训中引入大语言模型与 AI Agent 技术的现实出发点。

1.2.2 大语言模型在医学教育中的应用

近三年来，大语言模型在医学教育领域的应用研究迅速增长。国内研究已经从教学模式、能力培养、风险防控等角度讨论了 ChatGPT 类生成式人工智能对医学教育的影响^[4]；中文医疗大模型研究则进一步围绕医疗咨询、医学影像、心理健康等任务梳理了中文医疗大模型的发展脉络与评测挑战^[5]。在模型实践层面，HuatuoGPT^[6]通过医学问答数据和医生真实数据增强模型的医疗咨询能力，Qwen^[7]系列模型则在大规模中英双语语料和指令对齐数据上进行预训练，提供了通用与医学场景兼具的开放接口。国外研究中，Singhal 等^[8]和 Kung 等^[9]也分别从临床知识编码和医学考试评测角度验证了大语言模型辅助医学教育与临床培训的潜力。

上述工作共同验证了大语言模型作为可对话、可知识检索、可生成反馈的“虚拟带教教师”或“虚拟病人”的可行性。但是相关应用仍存在三方面的不足：第一，多数工作以“问答评测”为主要研究范式，缺乏围绕完整医学教学流程（问诊—查体—检查—诊断—治疗—反馈）的端到端系统设计；第二，单纯依赖提示词工程的应用方案在模型角色守界、客观医学事实控制与多轮对话记忆管理上较为脆弱，容易出现角色混淆与临床信息编造；第三，模型输出与教师教学评估、学生学情数据之间的闭环尚未形成，难以支撑医学院校的常态化教学场景。这些问题催生了在大语言模型之上构建工程化 Agent 编排层的研究路径。

1.2.3 AI Agent 与多智能体协作研究现状

将大语言模型扩展为具备规划、记忆与工具使用能力的“AI Agent”是大模型应用工程化的关键方向。Yao 等^[11]提出的 ReAct 框架将“推理（Reasoning）”与“行动（Acting）”在同一思维链中交错执行，使大模型能够在多步推理过程中动态调用外部工具；Schick 等^[12]的 Toolformer 进一步证明大模型能够通过自监督的方式学习何时与如何调用工具。Lewis 等^[13]提出的检索增强生成（RAG）则通过从外部知识库检索证据来约束生成内容，缓解大模型的幻觉与知识时效性问题。这三项工作共同奠定了“提示 + 检索 + 工具调用”这一现代大模型应用工程的基础范式。

在多智能体协作方向，Wu 等^[14]提出的 AutoGen 与 Hong 等^[15]提出的 MetaGPT 通过 Agent 之间的角色分工与对话协议解决复杂任务的协作执行问题；Wang 等^[16]的综述系统总结了基于大语言模型的自主 Agent 在感知、规划、记忆、行动等方面的研究脉络，并指出领域专用 Agent（如医疗、教育、金融）将成为未来重要的应用方向。本课题正是在上述研究背景之下，将多 Agent 协同、RAG 与工具调用引入医学问诊实训场景，针对“标准化病人扮演—诊断评估—治疗反馈”等差异化任务设计专用 Agent，并通过路由器在不同教学阶段进行动态编排。

1.3 研究目标与主要工作

本课题的研究目标可以概括为以下三点：

第一，面向医学院校问诊实训需求，设计并实现一套覆盖学生、教师、管理员三类角色的完整 Web 化模拟诊疗系统，提供 AI 问诊训练、病例库管理、学习分析与系统管控等核心功能。

第二，在通用大语言模型之上，构建包含 Agent 编排、知识检索、会话记忆、临床工具与结构化评分的医疗 AI 模拟诊疗框架，提升问诊训练的可控性、客观性与教学反馈质量。

第三，完成系统的容器化部署与全面测试，从功能、性能、安全等多个维度对所提方案的可行性与有效性进行验证。

围绕上述研究目标，本文的主要工作包括：

（1）需求分析与总体架构设计。系统化梳理医学问诊实训的真实业务流程，提炼三类用户的核心需求，给出基于前后端分离与多 Agent 协同的整体架构、数据库 ER 模型与接口规范。

（2）多 Agent 路由的对话引擎实现。抽象 PatientAgent、DiagnosisAgent、Feedback-Agent 三类 Agent 角色，由 AgentRouter 在不同训练阶段动态选择，避免在问诊中出现角色混淆与答案泄露。

（3）轻量化 RAG、滑动窗口 Memory 与临床工具自动注入机制实现。将医学知识、

对话历史与客观检查结果以工程化方式注入提示词，规避大模型在客观医学事实上的幻觉风险。

（4）多维结构化评分与规则兜底实现。以 JSON Schema 约束模型输出，给出问诊、诊断、治疗、沟通四个维度的可视化分析与改进建议，并在模型返回异常时通过规则兜底保障评分服务可用。

（5）端到端工程实现与部署测试。基于 Vue 3 与 Spring Boot 完成全栈实现，通过 Docker Compose 完成容器化部署，并从功能、性能、安全维度进行测试与验证。

1.4 论文组织结构

本论文围绕“虚拟问诊系统设计与实现”展开，全文共分为八章，按照“问题背景—相关技术—需求分析—架构设计—核心实现—模块详解—测试部署—总结展望”的逻辑层层推进。各章主要内容如下：

第 1 章绪论。介绍课题的研究背景与意义，综述国内外在医学教育数字化、大语言模型医学应用与 AI Agent 三个方向的研究现状，并明确本文的研究目标、主要工作与组织结构。

第 2 章相关技术概述。介绍系统涉及的关键技术，包括 Vue 3 前端框架、Spring Boot 后端框架、阿里百炼大语言模型平台、AI Agent 与多智能体协作、检索增强生成、Function Calling 工具调用、MySQL/Redis/MinIO 等数据基础设施以及 Docker 容器化部署。

第 3 章系统需求分析。从业务、功能与非功能需求等多个角度对系统需求进行系统化分析，给出三类用户的用例图与典型用例规约。

第 4 章系统总体设计。阐述系统的整体架构、功能模块划分、数据库 ER 模型、接口设计、RBAC 权限模型与安全架构，为后续的详细实现奠定基础。

第 5 章 AI Agent 核心技术实现。作为本论文的核心创新章节，重点呈现多 Agent 路由、RAG 知识检索、Redis 滑动窗口会话记忆、临床工具自动注入、结构化评分以及 SSE 流式响应等关键模块的设计原理与代码实现。

第 6 章系统功能模块详细实现。详细介绍学生端、教师端、管理员端及公共模块的关键页面、交互流程与代码实现，覆盖 AI 问诊训练、病例管理、批改工作台、学习分析等业务场景。

第 7 章系统测试与部署。给出功能、性能、安全与 AI Agent 效果验证的测试方案与结果，并描述系统基于 Docker Compose 的容器化部署过程与运行效果。

第 8 章总结与展望。对全文工作进行总结，凝练创新点，分析现有不足并给出后续研究方向。

第 2 章 相关技术概述

2.1 前端开发技术

2.1.1 Vue 3 与 Composition API

Vue.js 是尤雨溪于 2014 年推出的渐进式 JavaScript 前端框架，以响应式数据绑定、组件化开发与低从业门槛著称^[17]。本系统采用的 Vue 3.4.21 以 Proxy 重写了响应式系统，相较于 Vue 2 的 `Object.defineProperty` 方案在数组与动态属性跟踪上更为精准，运行时开销也明显下降。Composition API 是 Vue 3 推出的新一代逻辑组织方式，允许开发者将同一业务关心点的响应式状态、生命周期与方法集中在一个 `setup` 函数中，避免了 Options API 在复杂页面中逻辑被切分到 `data`、`methods`、`computed` 等选项中的问题。本系统的训练主页 `Training.vue` 页面超过 1900 行，包含 Agent 状态、对话流、计时器、表单提交等多个响应式状态与独立逻辑块，Composition API 可以将其拆分为可复用的复合函数，显著提升可读性与可维护性。配合 Vue Router 进行路由管理、Pinia 进行状态集中化管理、Vite 作为构建与开发服务器，构成了完整的现代前端工程体系。

2.1.2 Element Plus 与 ECharts

Element Plus 是面向 Vue 3 的企业级 UI 组件库，提供表格、表单、对话框、消息、上传、菜单等 60 多种高质量组件，不仅能够统一中后台系统的视觉语言，还内置了表单校验、表格分页、主题定制等业务能力。本系统的三个角色门户（学生、教师、管理员）都基于 Element Plus 2.6.3 构建，并通过 CSS 变量为不同角色提供独立的主题色与品牌语言。

ECharts 是国内广泛使用的声明式 Web 可视化框架^[18]，支持上百种图表类型、GPU 加速与大数据量增量渲染，在商业智能与数据可视化领域应用广泛。本系统使用 ECharts 5.5.0 实现了多种业务图表：雷达图用于呈现学生在问诊、诊断、治疗、沟通四个能力维度上的表现；折线图用于追踪训练成绩趋势；柱状图与饼图用于知识点掌握度与常见错误分布统计，为学生与教师提供可理解、可复盘的学情证据。

2.2 后端开发技术

2.2.1 Spring Boot 与 Spring Security

Spring Boot 是在 Spring Framework 之上构建的“约定优于配置”的快速开发框架，通过自动装配机制大幅减少了传统 Spring 项目中繁琐的 XML 与 Java Config 代码^[19]。本系统采用 Spring Boot 3.3.6，需要 Java 17 作为运行时环境；Spring Boot 3 系列全面迁移至 Jakarta EE 9+ 命名空间，并提供了原生镜像、可观测性与 WebFlux 响应式编程等特性，为 SSE 流式接口与高并发 AI 调用提供了原生支撑。

Spring Security 是 Spring 体系下的安全框架，提供身份认证、权限控制、会话管理、密码加密、CSRF 防护等能力。本系统的 SecurityConfig 采用无状态会话策略，在过滤器链中插入自定义的 JwtAuthenticationFilter，负责从 HTTP 头提取 Token、验证签名、加载用户信息并填充认证上下文；同时在 authorizeHttpRequests 中配置了登录、验证码、文件预览与文档入口等公开路径，其余接口需付与认证后访问，实现了轻量但清晰的安全边界。

2.2.2 MyBatis-Plus 与 JWT

MyBatis-Plus 是在 MyBatis 之上构建的增强型持久层框架，在保留 MyBatis 灵活 SQL 能力的同时，提供了 BaseMapper 通用 CRUD、LambdaQueryWrapper 类型安全查询、逻辑删除、自动填充、分页插件等开箱即用的能力^[20]。本系统采用 MyBatis-Plus 3.5.6 提供的 mybatis-plus-spring-boot3-starter 适配 Spring Boot 3，使用 LambdaQueryWrapper 表达复杂查询条件，使用分页插件实现训练记录列表与批改列表的高性能分页，并通过逻辑删除字段 deleted 保证业务数据的可追溯性。

JWT（JSON Web Token）是 IETF 在 RFC 7519 中定义的一种紧凑、安全、自包含的令牌传输规范^[21]，令牌由 Header、Payload、Signature 三部分经 Base64URL 编码拼接而成。与传统 Session-Cookie 机制相比，JWT 具有无状态、跨域友好、便于水平扩展的优势。本系统使用 JJWT 库实现令牌的签发与验证，Payload 中携带用户 ID、用户名与过期时间，并通过 Redis 存储刷新令牌；同时配合 LoginAttemptService 进行登录失败锁定，在保留无状态优势的同时弥补了传统 JWT 不可吊销的短板。

2.3 大语言模型与阿里百炼平台

大语言模型一般遵循“大规模预训练 + 指令微调 + 人类偏好对齐”的训练范式，通过在海量语料上的自监督学习获得世界知识，再由人类标注者或反馈模型对其输出进行可控性与安全性的对齐。通义千问（Qwen）是阿里巴巴开源的中文友好型大语言模型

系列^[7], 其预训练语料覆盖中英双语代码、多科学知识 with 多轮对话, 并通过指令对齐获得了较强的指令遵循、工具调用与思维链能力。

阿里云推出的百炼大模型服务平台 (DashScope) 以 SaaS 形式提供通义千问系列模型的推理服务, 同时提供与 OpenAI Chat Completions 完全兼容的调用接口^[22], 使开发者可以复用 OpenAI SDK 生态快速接入。本系统采用 qwen-plus 作为默认主模型, 在生产环境下可切换为响应更快的 qwen3.5-flash, 以平衡生成质量与推理延迟。后端通过 Spring WebFlux 提供的 WebClient 调用 OpenAI 兼容端点, 以 Bearer Token 身份验证 API Key, 并通过设置 stream=true 开启字粒级的 SSE 输出。调用参数中, temperature 由不同 Agent 的 AgentProfile 动态提供, max-tokens 设为 4096, 超时阈值为 120 秒, 以适应医学多轮对话的长上下文场景。

2.4 AI Agent 与多智能体协作

从能动主义 AI 的视角看, Agent 是一个可以感知环境、根据目标进行思考与决策、并采取动作以改变环境的实体。在大语言模型时代, Agent 被重新表述为“以 LLM 作为推理与控制核心、辅以记忆模块、规划模块与外部工具调用能力的软件实体”。Yao 等^[11]提出的 ReAct 框架是其中最具代表性的范式之一, 它要求模型以“思考 (Thought) — 行动 (Action) — 观察 (Observation)”的交错形式输出推理过程与动作调用, 并依据环境反馈逐步调整策略。

在多智能体协作方面, Wu 等^[14]提出的 AutoGen 以及 Hong 等^[15]提出的 MetaGPT 都采用了角色拆分与对话协议的思路: 不同 Agent 被赋予独立的系统提示词、工具集与参数设置, 在统一的调度器下进行多轮对话以完成一个复杂任务。Wang 等的综述^[16]进一步将该思路抽象为“感知—记忆—规划—行动”的四要素模型, 并指出领域专用 Agent 是未来重要的研究与应用方向。

本系统的 Agent 编排层正是在上述思路下针对医学问诊场景的专用实现: PatientAgent 负责在问诊阶段扮演标准化病人, 严格限制在病例信息范围内作答; DiagnosisAgent 负责在提交阶段对学生诊断思路进行启发式点评; FeedbackAgent 负责输出治疗反馈与风险提醒。三者由 AgentRouter 根据训练阶段与消息类型动态选择, 避免了“一个提示词同时负责扮演与评估”带来的角色混淆。

2.5 检索增强生成 RAG

大语言模型虽然掌握了广泛的世界知识, 但其知识受限于预训练语料的截止时间, 并且在生成过程中容易对不确定的事实“拍脑袋”, 即所谓的幻觉 (hallucination) 问题。Lewis 等^[13]提出的检索增强生成 (Retrieval-Augmented Generation, RAG) 框架在生成之前先从外部知识库中检索与查询相关的证据文档, 并将其拼接到提示词中作为上下

文。该范式可以在不修改模型参数的前提下，及时引入领域知识，从而明显缓解幻觉与知识时效性问题。

经典的 RAG 系统一般由“向量化—向量检索—重排—上下文拼接”几个阶段构成，其核心依赖于预训练的语义嵌入模型与高性能向量数据库（如 Milvus、Faiss、pgvector）。但在医学问诊实训场景中，知识点的总量中等（百条量级）、主题高度结构化、查询词多为主诉与所查询症状名词，不需要复杂的语义检索。本系统的 KnowledgeRAGService 采用轻量化的关键词匹配方案：先按中英文标点切分查询文本，保留长度不小于 2 的 token，取长度最长的 3 个关键词；随后在 MySQL 的知识点表中按“标题 LIKE % 词%”与“标签 LIKE % 词%”进行多列模糊查询，取 Top-3 结果拼接为“【相关知识点】”上下文。这种设计避免了向量库的运维成本与冗余依赖，在医学医诊实训场景下能够提供足够的检索质量。

2.6 工具调用 Tool Calling / Function Calling

工具调用（Tool Calling，也称 Function Calling）是让大语言模型跨出语言边界、可靠地访问外部世界与结构化服务的关键能力。Schick 等^[12]提出的 Toolformer 首次以“自监督方式教会模型”在哪里插入工具调用、调用什么、以及如何使用返回结果”。OpenAI 随后以 `tools` 参数与 `tool_calls` 返回字段将这一能力产品化，并被众多提供商采纳为事实上的行业标准。该规范要求使用 JSON Schema 描述每个工具的名称、说明与参数表，模型输出中以结构化字段表明选中的工具与参数取值，从而避免了传统让模型输出原始代码片段后再解析的不稳定性。

本系统的 AiToolService 严格遵循 OpenAI Function Calling 格式定义了三个医学领域专用工具：`query_symptom_knowledge` 用于查询症状或疾病的知识条目，参数为关键词与可选分类；`get_diagnosis_criteria` 用于获取疾病的诊断标准，参数为疾病名称；`evaluate_treatment_plan` 用于在训练结束时记录治疗方案以便后续评估，参数为诊断与治疗方案。每个工具的实现均使用 KnowledgePointMapper 在知识点表上进行模糊查询并返回结构化结果。考虑到本科系统的稳定性优先，配置项 `enableTools` 默认关闭，作为可选能力为后续向更复杂任务规划演进预留扩展点。

2.7 数据库与缓存技术

2.7.1 MySQL 8.0

MySQL 是当前互联网业务中使用最广泛的开源关系型数据库，本系统采用 MySQL 8.0 作为主业务数据存储，采用 InnoDB 存储引擎以获得事务与行级锁支持，采用 `utf8mb4` 字符集以覆盖完整 Unicode 与 Emoji。在表结构设计上，本系统充分利用了

MySQL 8.0 对 JSON 数据类型的原生支持: 病例表 `case_info` 中的 `medical_images` 字段存放临床影像附件信息, 评分表 `training_score` 中的 `weak_points`、`error_patterns`、`score_detail` 等字段存放结构化评分结果, 避免了为动态评分维度额外建表的冗余。

2.7.2 Redis 7

Redis 是高性能的内存型键值存储系统^[23], 支持字符串、哈希、列表、集合、有序集合、位图、流等多种数据结构。本系统采用 Redis 7 作为多个关键场景的支撑: 一是认证会话与验证码的高速读写, 二是登录失败计数与账号锁定机制, 三是作为 Memory 模块的存储后端。ConversationMemoryService 以 `session:memory:{recordId}` 为键以拼接后的多轮对话文本为值, 设置 24 小时过期与最多 10 轮的滑动窗口, 仅需一条 String 存储即可完成会话上下文的跨进程共享与自动清理。

2.7.3 MinIO 对象存储

MinIO 是一款高性能、与 Amazon S3 API 完全兼容的开源对象存储服务^[24], 适合作为医学影像、多媒体资源与文档附件的统一存储后端。本系统部署了独立的 MinIO 实例作为业务文件存储, 包括病例中的胸片、CT、超声、心电图、化验单等多种医学影像, 以及用户头像与批改附件。为应对内网上传与公网预览的场景, 系统在配置中区分了内部 `endpoint` 与公网 `public-endpoint`, 后端在上传后返回可跨网访问的预签名 URL, 避免了跨网络上下以及跨域访问的问题。

2.8 容器化部署技术

Docker 是一种基于 Linux 内核命名空间与控制组机制的轻量级虚拟化技术^[25], 通过镜像封装应用及其依赖, 在开发、测试与生产环境中提供一致的运行体验, 是云原生体系的基石。Docker Compose 是官方提供的多容器编排工具, 通过一份 `docker-compose.yml` 描述服务拓扑、镜像、端口、环境变量、网络与数据卷, 可以一键启动一整套服务。

本系统的 `docker-compose.yml` 编排了五个服务容器: MySQL 作为业务数据库、Redis 作为缓存与 Memory 后端、MinIO 作为对象存储、后端服务运行 Spring Boot 应用、前端服务运行 Nginx 代理 Vue 静态资源。服务之间通过自定义的 `ai-medical-network` 桥接网络进行内部联通, 并通过 `depends_on` 与 `healthcheck` 保证启动顺序。所有敏感参数(数据库密码、AI API Key、邮件服务密码等)以 `.env` 文件注入环境变量, 避免硬编码与仓库泄露。该架构使得全栈可以在任意联网主机上以一条命令完成部署。

2.9 本章小结

本章从前端、后端、大语言模型、Agent 与多智能体协作、检索增强生成、工具调用、数据与存储、容器化部署八个维度梳理了本系统所依赖的关键技术栈，重点说明了各项技术的选型动机及其在本系统中的具体作用。这些技术为后续章节中需求分析、总体设计以及以 Agent 为核心的详细实现提供了必要的理论与工程基础。接下来的第 3 章将在此基础上进一步对系统需求进行系统化的分析与梳理。

第 3 章 系统需求分析

3.1 业务需求分析

3.1.1 医学问诊实训现状与痛点

医学问诊是临床能力培养中不可跳过的入门环节,临床医学、护理、口腔医学等专业的本科生与实习医生都需要进行大量重复性、场景化的问诊训练。但在当前高校实训体系中,传统的班级课堂仅能提供表面化的角色扮演,临床带教老师亦受限于临床工作负荷,难以在课外多轮次地为每个学生提供个性化问诊指导。综合调研与项目需求分析,现有医学问诊实训主要存在以下四个痛点:

痛点一:真实的医学场景与标准化病人资源匮乏。传统课堂中由同学扮演病人,其临床表现与语言表达难以达到标准化病人的质量,而职业化标准化病人的培训与聘请产生较高的成本。痛点二:教师批量指导效率低下,个性化反馈难以被保证。一名临床带教教师往往需同时负责十几个甚至几十个学生的问诊指导与报告批改,几乎难以逐个逐轮点评。痛点三:实训效果难以量化评估,能力薄弱点不能及时发现。现行实训成绩多为教师主观评分,缺少从问诊、诊断、治疗、沟通多维度进行细粒度分析的数据集。痛点四:学生缺乏 24×7 的复课机会,实训过后难以在课外以低成本、低门槛的方式进行反复练习与查漏补缺。上述痛点交织在一起,构成了本系统需要重点解决的核心业务问题。

3.1.2 用户角色分析

结合系统部署后面向的使用人群与他们的职责划分,系统设定了五类角色键(role_key),在权限粒度上可进一步划分为三大使用面。学生(student)是系统最主要的使用者,包括临床医学、护理、口腔医学等专业的本科生与实习生,其核心需求是“多场景反复练习 + 可量化的能力反馈”。教师面包括临床带教老师(teacher)与教务管理员(teacher_admin)两类角色,后者在前者的基础上拥有跨班级、跨教师的数据查看与管理权限。管理员面包括系统管理员(admin)与超级管理员(super_admin),负责人、角色、权限、日志与系统资源的集中管控。表 3-1 总结了三类角色的描述与核心需求。

表 3-1 系统用户角色与核心需求对比

角色	描述	核心需求
学生	医学生 / 实习生	选择病例开启问诊训练、进行多轮 AI 对话与临床检查、提交诊断与治疗方案、查看评分与能力雷达、复盘训练记录、查看个人薄弱点与趋势分析
教师	临床带教老师 / 教务管理员	创建与维护医学病例库、上传多媒体临床资料、管理班级与学生名单、查看学生 AI 问诊全过程、在 AI 评分基础上补充教师评分与评语、查看班级能力雷达与错误点分布
管理员	系统管理员 / 超级管理员	管理用户与角色、重置密码与冻结账号、查看系统运行状态与在线人数、审查操作日志与异常事件、维护存储与配置

3.2 功能需求分析

3.2.1 学生端功能需求

学生端作为系统的主要使用面，需要提供以“AI 问诊训练”为中心、覆盖学习闭环的完整功能。其中最核心的是 AI 问诊训练模块，由 `TrainingController` 与 `ChatController` 联合提供，包括启动训练、多轮流式对话、临床检查调用、提交诊断与治疗方案、结束训练与放弃训练等子功能。考虑到问诊过程可能被意外中断，本系统还需支持“训练续接”能力：用户重新进入未结束的训练记录后，系统可以从 `Memory` 中恢复对话上下文，从中断点继续问诊。

除训练主流程外，学生端还需提供以下辅助功能：一是病例库浏览与详情查看，支持按科室、年龄、难度、临床重点进行筛选；二是训练记录与评估报告，由 `/api/training/list` 提供分页查看，包含总分、四维能力、诊断与治疗反馈、学习建议与薄弱点提示；三是学习分析仪表盘，由 `/api/analysis/student` 供给问诊、诊断、治疗、沟通四维能力雷达、训练成绩趋势、错误高发知识点、个性化推荐病例；四是个人中心，允许修改资料、上传头像、修改密码与查看通知。在交互设计上需以轻量、丰富与面向年轻学生群体的视觉表达为主，与其他职业使用者门户在设计语言上区分开。

3.2.2 教师端功能需求

教师端是系统中仅次于学生端的关键使用面，其需求可抽象为“资源供给 + 过程监看 + 结果反馈”三个层面。资源供给层面表现为病例管理能力：教师需能够创建、编

辑、复制、发布与归档病例，为每个病例设置主诉、现病史、既往史、体格检查、临床检查、诊断与治疗方案等字段，上传胸片、CT、超声、心电图、化验单等附件，并为不同难度、不同科室设置标签。

过程监看层面表现为班级管理 with 训练记录查询：教师需能够创建班级、添加学生、查看班级中学生的训练记录 with 对话详情，跨班级、跨教师的数据查询能力仅允许 `teacher_admin` 以上角色访问。结果反馈层面表现为成绩 with 批改工作台：由 `GradeController` 提供成绩列表、详情、单条评分、批量评分、班级统计 with 导出能力，在 AI 自动评分的基础上补充教师评分 with 个性化评语，并通过 `AnalysisController` 提供班级整体的能力雷达、错误点分布等教学分析报表，帮助教师从微观个体维度 with 宏观班级维度同时评估教学效果。

3.2.3 管理员端功能需求

管理员端面向系统管理员 with 超级管理员，负责为学生端 with 教师端提供稳定、安全、可追溯的运行环境，需求主要集中在人、权、日、资四个方面。人（用户）方面，需要提供用户查询、创建、修改、删除（逻辑删除）、重置密码、冻结 with 解冻账号能力，并支持按角色 with 关键词检索。权（角色 with 权限）方面，需提供角色的创建、修改、删除 with 不同角色下用户数量的统计，并为后续菜单 with 接口级权限预留扩展点。日（日志）方面，需提供操作日志、异常日志的检索 with 导出能力，记录请求路径、参数、耗时、错误信息以保障可追溯性。资（系统资源）方面，需提供 CPU、内存、磁盘、运行时间、在线用户数、业务总量等实时指标的可视化呈现，辅以 MinIO 桶 with 配额、Redis 连接状态等运维辅助能力。

3.2.4 公共与核心技术需求

除三类角色的个性化需求外，系统还需提供由多个角色共享的公共能力。一是身份认证 with 权限控制能力，需支持邮箱 + 验证码注册、账密 with 邮箱验证码双重登录、登录失败锁定 with 验证码保护、密码重置 with 令牌刷新，并采用 RBAC 模型进行页面菜单、接口路径以及按钮三级粒度的权限控制。二是文件上传 with 预览能力，面向医学影像、护理文档、用户头像 with 批改附件等场景，需提供安全上传、预签名 URL、预览 with 下载能力。三是数据可视化能力，需以雷达图、折线图、柱状图、饼图进行多维度呈现，为三类角色提供一致的可视化体验。四是 AI 流式响应能力，需基于 SSE 或 WebSocket 提供字粒级的流式输出，以避免多轮对话中长时间的空白等待体验。这些公共能力为上述三个业务面提供统一、可复用的技术基础。

3.3 非功能需求分析

3.3.1 性能需求

本系统面向高校多校区、多班级同时在线使用的场景，需要在响应时间、并发能力与多媒体体验三个维度上满足以下指标。一是一般业务接口的服务端响应时间，在网络连通状况下要求不超过 2 秒，面向多表联合与班级统计等重型查询接口，要求不超过 5 秒。二是 AI 请求的首字返回时间，在不启用调用工具的条件下不超过 5 秒，在部分需 RAG 检索与上下文拼接场景下不超过 8 秒。三是并发能力，任务书要求系统支持 10000+ 同时在线用户，实际部署中通过多实例与负载均衡达成。四是多媒体体验，医学影像与附件上传不能明显卡顿，预览需后台提供缩略图以提升列表加载速度。表 3-2 总结了关键性能指标。

表 3-2 系统关键性能指标

指标维度	目标值	备注
一般接口响应时间	≤ 2 秒	单表查询、列表分页、详情查询
重型查询响应时间	≤ 5 秒	多表联合、班级统计、训练分析
AI 首字返回时间	≤ 5 秒	不启用工具调用
AI RAG 首字返回时间	≤ 8 秒	含知识检索与上下文拼接
并发在线用户	≥ 10000	任务书指标，以多实例部署为前提
附件上传大小上限	≤ 50 MB	医学影像与护理文档

3.3.2 安全需求

医学教育应用涉及学生身份信息、学习轨迹与考核成绩，最严重的泄露与越权需求高于传统企业管理系统。本系统需从身份、权限、数据、审计四个层面建立安全防线。身份层面，需采用密码 BCrypt 加盐存储、邮件验证码与图形验证码双重验证、JWT 令牌 + 刷新令牌双令牌机制，并在连续登录失败 5 次后锁定账号 30 分钟。权限层面，需采用 RBAC 模型的页面、接口、数据三级防越权：页面与菜单由前端路由守卫控制，接口由 Spring Security 过滤器控制，数据级别则由业务表中的 `teacher_id`、`class_id` 范围进行二次过滤，避免跨班级、跨教师的数据泄露。数据层面，需以 HTTPS 预设传输加密，对邮箱、手机号、身份证号等敏感字段采用 AES 存储加密，以参数化查询、Spring Security 默认 XSS 过滤与 CSRF Token 防范常见 Web 注入攻击。审计层面，需记录登录、重要写操作与权限变更的操作日志，提供可追溯的安全事件证据。

3.3.3 可用性与可维护性需求

在可用性上，前端需兼容主流的 Chrome 110+、Edge 110+、Firefox 110+ 与 Safari 16+ 浏览器，布局上采用响应式设计，在 1280~1920 px 桌面分辨率与 1024 px 以上的平板上需保证主要布局与交互路径可用。在可维护性上，后端采用多模块拆分与分层架构，代码采用资源、业务、数据三层表达，前后端接口遵循 REST 风格与统一响应体。同时需提供完整的 README、数据库初始化 SQL、Docker 部署说明与 API 接口文档，以便后续交接与迭代。

3.4 用例分析

为进一步可视化前述需求在不同角色上的分布与交互，本节采用 UML 用例图对系统总体进行抽象。图 3-1 展示了三类主参与者与系统核心用例的关系。学生作为主参与者可以参与“开始 AI 问诊训练”“查看训练报告”“查看学习分析”与“维护个人信息”等用例；教师可以参与“创建与发布病例”“管理班级与学生”“批改训练报告”与“查看教学分析”等用例；管理员可以参与“用户与角色管理”“查看系统监控”与“审查操作日志”等用例。此外，身份认证、文件上传与预览、消息通知等公用用例作为被多个主用例“include”的公共能力。

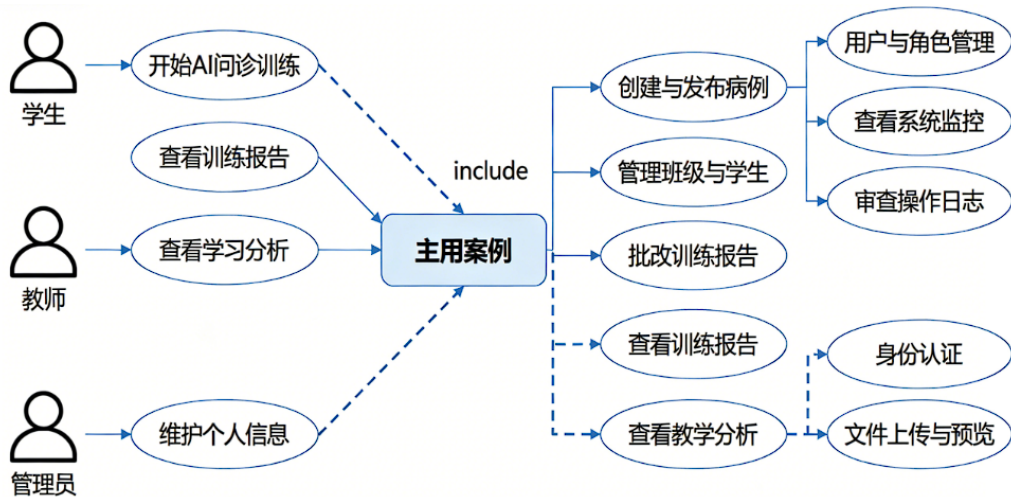


图 3-1 系统总体用例图

为体现详细的交互逻辑，本节选取“完成一次 AI 问诊训练”作为代表性用例进行规约描述，其详细内容如表 3-3 所示。该用例是学生端最高频、最重要的交互，也是背后联动 AI Agent、RAG、Memory、工具调用与评分最多的用例，能够集中呈现系统的核心业务逻辑。

表 3-3 用例规约：完成一次 AI 问诊训练

项目	内容
用例名称	完成一次 AI 问诊训练
主参与者	学生
次参与者	PatientAgent、DiagnosisAgent、FeedbackAgent、大语言模型服务
前置条件	学生已登录且账号状态正常，系统中至少存在一条已发布的病例
后置条件	生成一条训练记录与一条评分记录，并更新学生的学习分析指标
主流程	(1) 选择病例“开始训练”；(2) 后端创建训练记录，返回记录 ID；(3) 学生输入问诊问题，前端以 SSE 调用接口；(4) 后端由 AgentRouter 选择 PatientAgent，从 Memory 与 RAG 中拼接上下文，调用大语言模型生成回复并以流式返回；(5) 学生可重复步骤 (3)~(4) 进行多轮问诊，然后提交诊断与治疗方案；(6) 生成多维评分与学习建议。

为更直观地呈现上述用例的执行路径，图 3-2 以 UML 活动图的形式展示了学生完成一次 AI 问诊训练的完整业务流程。

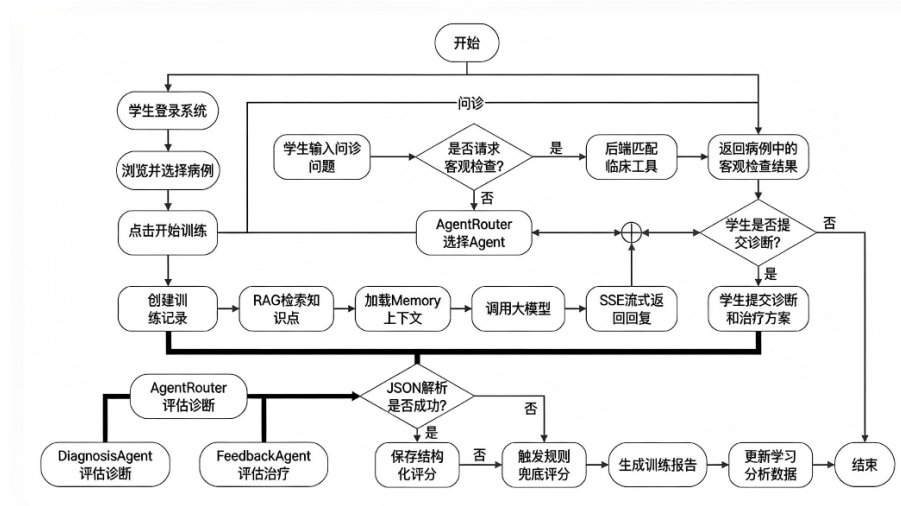


图 3-2 学生完成一次 AI 问诊训练活动图

3.5 本章小结

本章从医学问诊实训场景的业务痛点出发，梳理了学生、教师与管理员三类角色的使用需求，给出了以“AI 问诊训练”为核心、覆盖资源供给与运行运维的功能需求清单，并从性能、安全、可用性与可维护性四个维度给出了关键指标。最后通过总体用例图与核心用例规约表呈现了系统的交互边界，为第 4 章的架构设计提供了明确的需求输入。

第 4 章 系统总体设计

4.1 系统架构设计

4.1.1 整体架构

本系统采用“用户层—接入层—应用层—AI 服务层—数据层”五层架构进行设计，各层之间以标准化协议交互，职责边界清晰，便于独立迭代与水平扩展。系统整体架构如图 4-1 所示。用户层作为架构的最外层，包含学生、教师与管理员三类使用者，以主流浏览器为接入终端；接入层由 Nginx 反向代理服务构成，负责静态资源下发、路由分发、HTTPS 终结与跨域控制。应用层包括 Vue 3 前端应用与 Spring Boot 后端服务两个子部分：前端以静态资源形式部署于 Nginx，后端以 Spring Boot 应用运行在独立容器中，对外提供 RESTful API 与 SSE 接口。AI 服务层是本系统区别于传统教学系统的核心抽象，包含 Agent 路由、RAG 检索、会话记忆、工具调用与评分兜底五个模块，是介于应用层与外部大语言模型之间的领域中层。数据层采用 MySQL 存储业务数据、Redis 存储会话与缓存、MinIO 存储医学影像与文档附件，外部 AI 服务以 SaaS 方式调用阿里云百炼提供的通义千问模型。这种分层设计的优势在于：各层只依赖于下一层的对外接口，可以独立选择实现、独立部署；AI 服务层的抽象为后续替换模型、添加多模型路由与增加本地部署模型预留了扩展点。

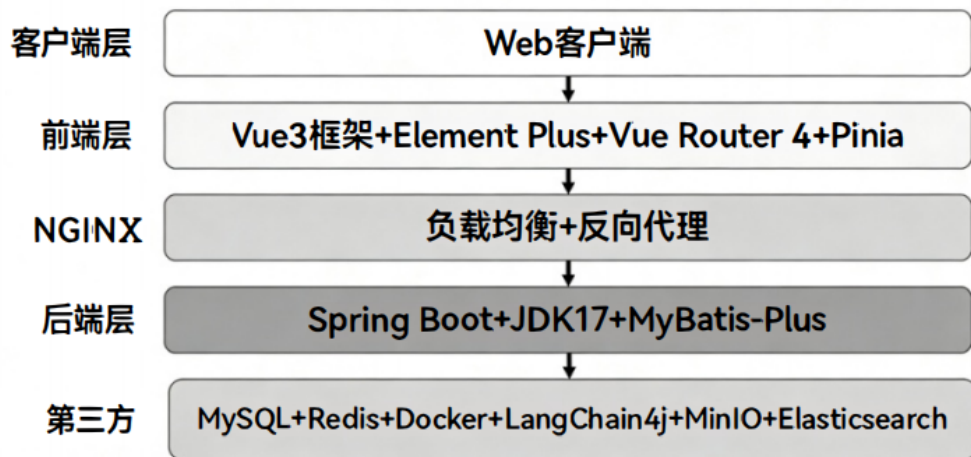


图 4-1 系统整体架构图

4.1.2 前后端分离架构

上述架构中的应用层采用严格的前后端分离设计。前端以 Vue 3 + TypeScript + Element Plus 为技术选型，使用 Vite 作为构建与开发服务器，所有页面在构建后以

dist/ 静态资源的形式交由 Nginx 托管。后端以 Spring Boot 3.3.6 为核心运行环境，对外提供两类接口：一是遵循 REST 风格的业务接口，以 `/api/<资源>/<动作>` 的路由约定与统一响应体返回业务数据；二是面向 AI 多轮对话场景的 SSE 流式接口，以 `text/event-stream` 进行字粒级推送。

前后端之间的路由交给 Nginx 处理：前端资源以 `/` 作为路径前缀，后端接口以 `/api/` 作为前缀转发到 Spring Boot 实例的 8081 端口，SSE 接口则将 `proxy_buffering` 关闭以保证实时推送。身份认证采用 JWT 令牌机制，前端在 `Authorization` 请求头中携带 `Bearer <token>`，后端通过过滤器进行验证与上下文注入。这种架构使前后端可以独立迭代与部署，同时为后续推出移动端、桌面端与开放 API 预留了扩展空间。

4.1.3 多 Agent 协同架构

为避免在同一个提示词中同时负责“扮演病人”与“评估学生”带来的角色混淆问题，本系统在大语言模型之上构建了一个轻量级的 Agent 编排层。如图 4-2 所示，学生发送的消息先进入 `AgentRouter`，路由器根据当前训练阶段与消息类型在三个领域 Agent（`PatientAgent`、`DiagnosisAgent`、`FeedbackAgent`）中选择一个最合适的实体；被选中的 Agent 提供专属的 `AgentProfile`（包含系统提示词与温度参数），并与三个横向能力模块联动：`KnowledgeRAGService` 从知识库中检索与话题相关的医学知识点；`ConversationMemoryService` 从 Redis 中恢复多轮上下文；`AiToolService` 在必要时以 `Function Calling` 形式调用临床工具。

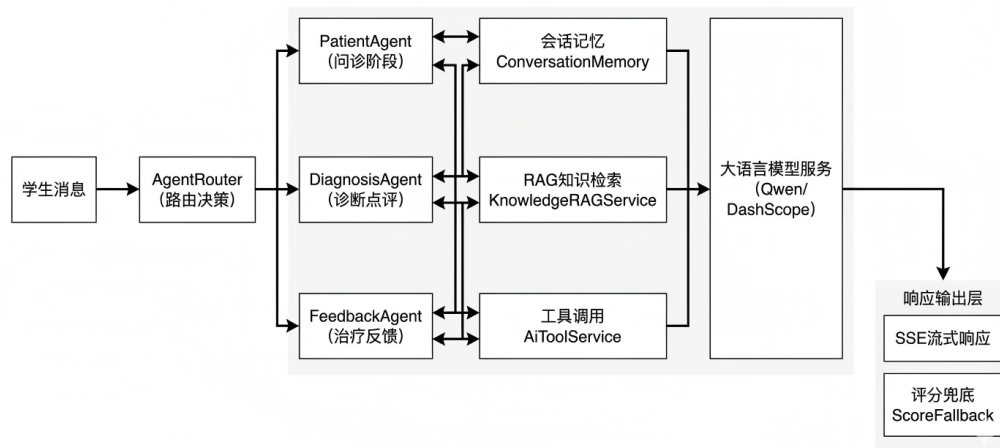


图 4-2 多 Agent 协同架构示意图

在获得上下文后，Agent 层将完整提示词提交给外部的大语言模型服务（DashScope / Qwen），后者以 SSE 流式响应返回生成结果。返回后的话术同时被三者消费：一是被转发给前端以实现逐字输出；二是被写入训练对话表（`training_dialog`）永久存储；三

是被追加到 Memory 中作为下一轮对话的上下文。在训练结束提交阶段，AgentRouter 切换到 DiagnosisAgent 与 FeedbackAgent，并调用 `AIService.evaluateTraining()` 以结构化 JSON 形式输出多维评分；若 JSON 解析失败则触发规则兜底（ScoreFallback），生成保底评分以保证服务可用性。该架构作为本系统的核心创新点，在第 5 章中将进行详细实现说明。

4.2 功能模块划分

系统功能模块按照“使用角色 + 公共能力 + AI 核心能力”的方式划分。学生端围绕训练闭环展开，教师端围绕病例资源与教学评价展开，管理员端围绕系统运行与权限治理展开，公共模块为三类用户提供统一认证、消息、文件与可视化能力，AI 核心模块则作为横向能力被学生训练、教师批改与学习分析复用。学生端的核心业务是完成一次可复盘的 AI 问诊训练，因此其模块包括病例浏览、训练启动、多轮问诊、诊断与治疗提交、训练报告、学习分析和个人中心。教师端负责教学资源供给与结果反馈，包括病例管理、班级管理、成绩管理、批改工作台、教学分析和通知管理。管理员端负责系统治理，包括用户管理、角色权限、系统监控、操作日志、存储管理和基础数据维护。公共模块包括 JWT 登录认证、验证码、文件上传、消息通知、统一异常与接口响应。AI 核心模块包括 Agent 路由、RAG 知识检索、Redis 会话记忆、临床工具调用和结构化评分。该划分方式保证了业务边界清晰，同时也便于后续按照模块进行独立开发、测试和部署。

4.3 数据库设计

4.3.1 ER 图

数据库设计以“用户—病例—训练—评分”为主线，并向教学班级、知识点、权限与日志扩展。核心实体包括用户表 `sys_user`、角色表 `sys_role`、病例表 `case_info`、病例内容表 `case_content`、训练记录表 `training_record`、训练对话表 `training_dialog`、训练评分表 `training_score`、知识点表 `knowledge_point`、班级表 `class_info` 及班级学生关联表 `class_student`。其中，用户与角色是一对多关系；教师用户创建病例，学生用户基于病例产生训练记录；每条训练记录可对应多轮对话与一条结构化评分记录；病例与知识点通过 `case_knowledge` 形成多对多关系，用于支撑 RAG 检索与学习薄弱点分析；班级与学生通过关联表建立多对多关系，用于教师端班级统计和成绩批改。系统核心数据库 ER 图如图 4-3 所示。

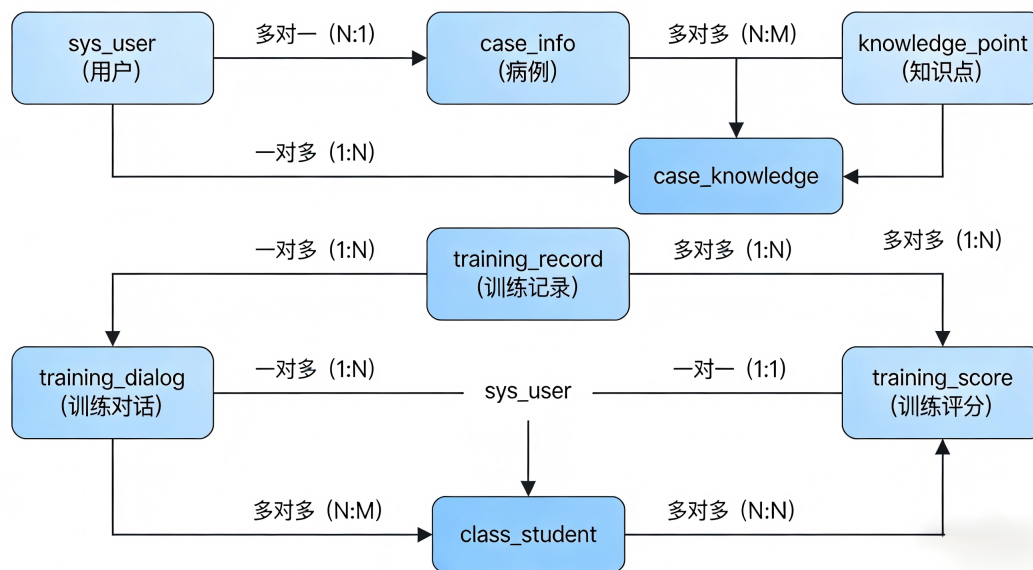


图 4-3 系统核心数据库 ER 图

为直观展示上述实体在一次问诊训练中的运行时关系，图 4-4 给出了一个典型训练场景的 UML 对象图。

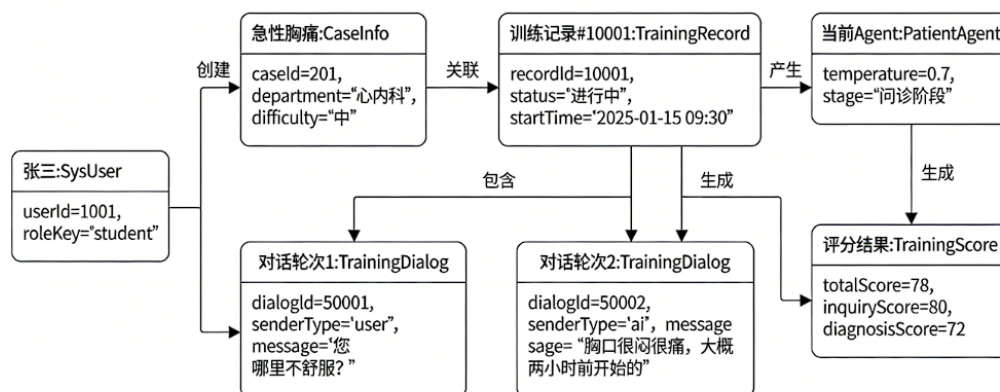


图 4-4 AI 问诊训练运行时对象图

4.3.2 主要数据表设计

系统采用 MySQL 8.0 作为核心业务数据库，字符集统一为 utf8mb4，以支持中文病例、医学术语与多语言文本。业务表普遍包含 create_time、update_time、deleted 等字段，便于审计和逻辑删除。考虑到本科毕业论文正文不宜堆叠过多字段明细，本节不逐表罗列全部字段，而是围绕系统主业务链路概括核心数据表的职责、关键字段和关系，如表 4-1 所示。完整建表语句可放入附录或随项目源码提交。

表 4-1 核心业务数据表汇总

数据表	主要职责	关键字段	关联关系
sys_user	用户身份与基础资料	登录名、密码哈希、角色、学号或工号、账号状态	多个用户对应一个角色
case_info	病例基础信息与 Agent 问诊依据	标题、科室、难度、主诉、参考诊断、参考治疗方案	教师创建病例，训练记录引用病例
training_record	一次问诊训练的主记录	学生、病例、训练状态、提交诊断、提交治疗方案	连接学生、病例、对话和评分
training_dialog	多轮问诊对话明细	训练记录、发送方、消息内容、消息类型、创建时间	一条训练记录对应多条对话
training_score	AI 与教师评分结果	总分、问诊分、诊断分、治疗分、薄弱点、评分版本	一条训练记录对应一组评分
knowledge_point	RAG 检索和学习分析知识库	标题、正文、分类、标签、错误次数	通过病例知识点关系与病例关联
class_info / class_student	班级管理 with 成绩统计范围	班级名称、专业、教师、学生、加入状态	教师按班级查看学生训练结果

4.3.3 Redis 数据结构

Redis 在系统中承担临时状态、登录安全与 AI 对话上下文缓存的职责。与 MySQL 中的持久化训练记录不同，Redis 数据均设置 TTL 或可主动清理，避免对话中间态长期堆积。表 4-2 给出了系统中的主要 Redis Key 设计。

表 4-2 Redis 主要数据结构设计

Key 模式	类型	过期策略	用途
session:memory:{recordId}	String	24 小时	保存某次训练最近若干轮对话上下文，供 Agent 拼接 Memory
captcha:{sessionId}	String	5 分钟	图形验证码校验，验证成功后立即删除
login:attempt:{username}	String / Number	30 分钟	登录失败次数统计，超过阈值后触发锁定
login:lock:{username}	String	30 分钟	账户临时锁定标记，用于防暴力破解
email:code:{email}	String	短期 TTL	邮件验证码，支撑注册和密码重置

4.4 接口设计

后端接口统一以 /api 作为业务前缀，采用 JSON 作为主要数据交换格式。普通业务接口返回统一结构 {code, message, data}，其中 code=200 表示业务成功，认证失

败返回 401，参数错误返回 400，系统异常返回 500。AI 流式对话接口因需要逐字输出，使用 `text/event-stream` 作为响应类型，前端通过 `EventSource` 或流式请求解析增量内容。

表 4-3 核心接口分组设计

接口分组	代表路径	主要职责
认证接口	<code>/api/auth/*</code>	登录、注册、验证码、刷新令牌、退出、密码重置、用户信息维护
病例接口	<code>/api/case/*</code>	病例列表、病例详情、病例新增、编辑、删除、元数据和推荐病例
训练接口	<code>/api/training/*</code>	开始训练、结束训练、放弃训练、训练列表、训练详情、提交评估
对话接口	<code>/api/chat/*</code>	普通发送、SSE 流式对话、历史对话查询
AI 接口	<code>/api/ai/*</code>	通用 AI 对话、流式输出、训练评分评估
成绩接口	<code>/api/grades/*</code>	成绩列表、教师评价、批量评分、学生与班级统计、成绩导出
分析接口	<code>/api/analysis/*</code>	学生学习分析、教师教学分析、雷达图与趋势统计
管理接口	<code>/api/admin/*</code>	用户、角色、日志、监控、存储和通知管理
文件接口	<code>/api/files/*</code>	文件上传、对象预览、医学影像访问

以“发送一次 AI 问诊消息”为例，前端在已登录状态下调用 `POST /api/chat/send`，请求头携带 JWT，主体中包含训练记录编号、病例编号与用户消息。若调用 `POST /api/chat/stream/{recordId}`，后端会在校验训练归属后进入 Agent 编排流程，并通过 SSE 逐段返回 AI 回复。非流式接口的典型请求与响应结构如下：

Listing 4.1 AI 问诊消息接口请求与响应示例

```
1 POST /api/chat/send
2 Authorization: Bearer <token>
3 Content-Type: application/json
4
5 {
6   "recordId": 10001,
7   "caseId": 1,
```

```
8  "message": "请问您胸痛是从什么时候开始的？"
9  }
10
11 {
12  "code": 200,
13  "message": "操作成功",
14  "data": {
15    "recordId": 10001,
16    "senderType": "ai",
17    "message": "大概两个小时前开始的，突然胸口很闷、很痛。",
18    "messageType": "text"
19  }
20 }
```

4.5 RBAC 权限模型设计

权限模型采用经典 RBAC（Role-Based Access Control）思想，通过“用户—角色—权限”三层关系降低授权复杂度。当前数据库初始化脚本已经落地 `super_admin`、`admin`、`teacher`、`student` 四类角色，其中超级管理员拥有全局权限，管理员负责用户、日志和系统资源管理，教师负责病例、班级、成绩与教学分析，学生负责病例训练与个人学习分析。任务书与需求分析中提到的 `teacher_admin` 可作为后续扩展角色，用于跨班级、跨教师的教务管理场景。

权限粒度按菜单、按钮、接口三层设计。菜单权限决定前端路由是否可见，按钮权限控制页面内“新增、编辑、删除、导出、批量评分”等具体操作，接口权限由后端 Spring Security 与业务层二次校验共同完成。对于训练记录、班级成绩等存在数据归属关系的资源，系统还需在业务层根据 `user_id`、`teacher_id`、`class_id` 进行数据范围限制，避免同级角色之间相互越权。

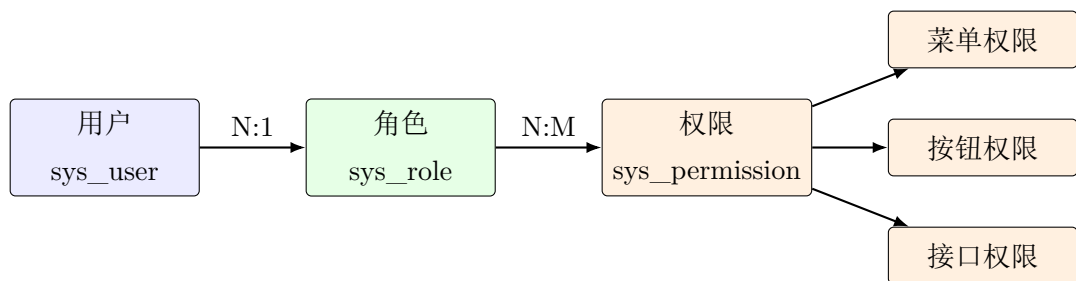


图 4-5 RBAC 权限模型设计

4.6 安全架构设计

系统安全架构从身份认证、访问控制、数据保护、输入校验和审计追踪五个层面设计。身份认证层采用 JWT 令牌机制^[21], 后端通过 `JwtAuthenticationFilter` 从 `Authorization` 请求头中解析 `Bearer` 令牌, 校验通过后将用户信息写入 Spring Security 上下文。登录入口叠加图形验证码与登录失败锁定机制: 验证码以 `captcha:{sessionId}` 存入 Redis, 5 分钟后自动过期; 同一用户名连续失败 5 次后写入 `login:lock:{username}`, 锁定 30 分钟, 以降低暴力破解风险。

访问控制层采用 Spring Security 的无状态会话策略, 登录、注册、验证码、健康检查、文件预览等少数路径开放访问, 其余接口默认要求认证。对于学生训练记录、教师班级成绩和管理员数据, 系统在 Controller 之后的 Service 层继续校验资源归属, 形成“认证 + 角色 + 数据范围”的组合防护。数据保护层对密码采用 BCrypt 单向哈希存储, 对手机号、邮箱等敏感字段预留 `@Encrypted` 注解与 `AESUtil` 字段级加密能力; 操作日志切面 `@OperationLog` 会对密码、token、验证码等敏感参数进行脱敏后写入 `operation_log`。输入与接口层面, 系统通过 DTO 校验、统一异常处理、MyBatis-Plus 参数化查询以及 Nginx 跨域策略降低 SQL 注入、越权调用与异常信息泄露风险。由于系统主要面向前后端分离的无状态 API, 当前后端关闭 CSRF, 并以 JWT、CORS 白名单和接口鉴权共同保障调用安全。

4.7 本章小结

本章在需求分析的基础上完成了系统总体设计, 给出了分层架构、前后端分离方式、多 Agent 协同架构、功能模块划分、核心数据库模型、Redis 缓存结构、接口分组、RBAC 权限模型和安全架构。总体上, 本系统并非简单地把大语言模型接入传统教学系统, 而是在 Web 工程架构中单独抽象出 AI 服务层, 使 Agent、RAG、Memory、工具调用和评分能力可以被训练、批改与学习分析复用。下一章将进一步展开 AI Agent 核心技术实现, 说明系统如何将大语言模型改造为可控、可追踪、可评分的医学问诊训练引擎。

第 5 章 AI Agent 核心技术实现

本章围绕系统中最具创新性的 AI Agent 层展开说明。与传统“单提示词 + 单轮问答”的大模型应用不同，本系统在 Spring Boot 后端中单独抽象出 Agent 编排层，并将病例信息、RAG 知识检索、Redis 会话记忆、临床工具自动注入和结构化评分组合为一条完整的训练流水线。该设计参考了 ReAct 中“推理与行动协同”的思想^[11]、Toolformer 对工具调用能力的讨论^[12]以及 RAG 对外部知识注入的基本范式^[13]，但实现上结合本科毕业设计的工程规模，采用轻量、可部署、可解释的方式完成。

5.1 多 Agent 路由架构

5.1.1 Agent 抽象与配置

本系统将 Agent 定义为“角色身份、显示名称、系统提示词与模型温度参数”的组合。后端通过 `AgentRouter.AgentProfile` 承载上述配置，避免在业务代码中直接散落大段提示词。该抽象虽然简单，但具有明确的工程价值：一方面，学生端问诊、训练提交后的诊断反馈、治疗方案反馈可以使用不同的角色边界；另一方面，后续如需新增安全审查 Agent、考试监考 Agent 或多模型路由器，只需扩展新的 Profile，而不需要重写聊天服务主流程。

Listing 5.1 Agent 配置抽象 AgentProfile（节选）

```
1 public static class AgentProfile {
2     private final String name;
3     private final String label;
4     private final String systemPrompt;
5     private final double temperature;
6
7     public AgentProfile(String name, String label,
8         String systemPrompt, double temperature) {
9         this.name = name;
10        this.label = label;
11        this.systemPrompt = systemPrompt;
12        this.temperature = temperature;
13    }
14 }
```

在当前版本中，系统包含 `PatientAgent`、`DiagnosisAgent` 与 `FeedbackAgent` 三类 Agent。其中 `PatientAgent` 负责训练过程中的标准化病人扮演，`DiagnosisAgent` 与 `Feed-`

backAgent 主要服务于提交后的诊断、治疗反馈和评分阶段。三个 Agent 共用同一套调用链路，但提示词、温度参数和业务约束不同。

5.1.2 PatientAgent：虚拟标准化病人

PatientAgent 是学生问诊训练阶段最重要的 Agent，其目标不是“教学生”，而是稳定地扮演病例中的患者。提示词要求模型严格使用患者第一人称作答，不主动给出诊断、鉴别诊断、治疗方案或标准答案；对于体格检查、化验、影像等客观检查结果，PatientAgent 只说明愿意配合检查，具体结果由系统临床工具从病例数据中返回。这一设计可以显著降低大模型在问诊过程中“提前泄题”或“主动教学”的概率。

Listing 5.2 PatientAgent 系统提示词（节选）

```
1 "你是一名虚拟标准化病人，学生是一名医生。"  
2 "请严格以患者第一人称回答医生的问题，只表现为病例中的患者本人。"  
3 "不要扮演教师，不要问诊引导，不要评价学生，"  
4 "不要提示下一步检查，不要主动给出诊断、鉴别诊断、治疗方案或标准答案。"  
5 "如果医生询问体格检查、化验、影像等客观结果，只说明你愿意配合检查；"  
6 "具体检查结果由系统临床工具返回。"
```

PatientAgent 的温度参数设置为 0.8，略高于诊断与治疗反馈 Agent，目的是让患者回答更自然、更口语化。医学问诊训练不仅要求学生识别医学事实，还要求他们适应患者真实表达中可能出现的模糊、犹豫和非专业描述，因此问诊阶段不应完全追求机械化的标准答案。

5.1.3 DiagnosisAgent：诊断评估

DiagnosisAgent 面向训练提交后的诊断评估场景，温度参数设置为 0.4。与 PatientAgent 相比，DiagnosisAgent 的输出需要更严谨、更结构化，重点评价学生是否识别关键阳性和阴性证据，是否给出合理的鉴别诊断，诊断依据是否与问诊结果和检查结果一致。其提示词强调“启发式反馈”，即指出学生推理链中的遗漏、跳步和逻辑漏洞，但不直接替学生完成全部诊断结论。

Listing 5.3 DiagnosisAgent 提示词设计（节选）

```
1 "你是一名诊断评估 Agent，负责围绕学生给出的诊断思路提供结构化反馈。"  
2 "请重点评估诊断依据是否充分、关键阳性与阴性证据是否被正确识别、"  
3 "鉴别诊断是否完整且有优先级，并指出推理链条中的遗漏、跳步或逻辑漏洞。"  
4 "不要直接替学生完成全部诊断结论，应保持启发式反馈。"
```

这种设计适合教学训练场景：如果 AI 直接给出完整答案，学生可能跳过临床推理过程；如果 AI 只做结果判断，又难以帮助学生发现薄弱环节。DiagnosisAgent 采用“问

题定位 + 证据提示 + 推理建议”的方式，在保证教学价值的同时降低答案泄露。

5.1.4 FeedbackAgent：治疗反馈

FeedbackAgent 面向治疗方案与综合反馈，其温度参数为 0.5，介于 PatientAgent 与 DiagnosisAgent 之间。该 Agent 的提示词要求围绕处理原则、治疗思路、风险提醒与随访建议展开，强调规范化临床思维和安全边界。由于本系统用于医学教学，不直接用于真实医疗决策，因此 FeedbackAgent 明确避免输出绝对化、唯一化或替代临床面诊的医疗指令；遇到高风险情况时，提示进一步评估或转诊。

在训练报告中，FeedbackAgent 的结果会与结构化评分共同构成学生的复盘材料。学生可以看到自己在问诊完整性、诊断准确性、治疗规范性和沟通表达方面的得分，也能获得下一次训练的具体建议。教师端则可以在 AI 反馈基础上进行人工复核和补充评价。

5.1.5 AgentRouter 路由策略

当前系统的 `AgentRouter.route()` 方法采用保守路由策略：学生训练过程中始终路由到 PatientAgent，训练提交后再由评分流程调用诊断和反馈能力。这样做的原因是，问诊阶段最重要的教学目标是让学生主动收集病史和检查线索，而不是让 AI 直接进入教学点评。一旦在问诊过程中动态切换到诊断或治疗反馈 Agent，就可能出现提前提示诊断方向、暴露参考答案的问题。

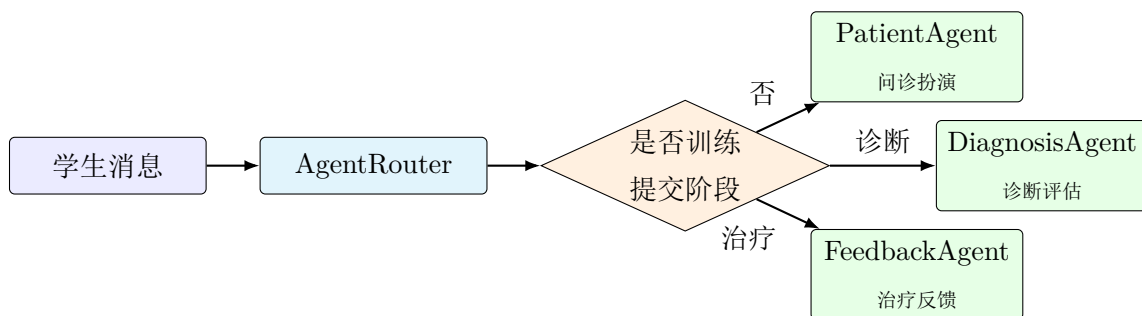


图 5-1 Agent Router 决策流程

该策略并不排斥后续扩展。未来可引入消息分类器、关键词规则或 Function Calling 结果来实现更精细的路由，例如将“我要提交诊断”路由到 DiagnosisAgent，将“请评价治疗方案”路由到 FeedbackAgent，将“这个回答是否安全”路由到 SafetyAgent。但在本科毕设版本中，稳定守住患者角色边界比追求复杂路由更重要。

5.2 RAG 知识检索增强

5.2.1 关键词提取与召回策略

医学问诊训练需要一定的外部知识支撑，但本系统没有采用向量数据库，而是实现了基于关键词匹配的轻量级 RAG。原因主要有三点：首先，当前病例与知识点数据量处于中小规模，知识点已被结构化存储在 `knowledge_point` 表中；其次，部署环境以 Docker Compose 为主，引入向量数据库会增加运维复杂度；最后，本系统的知识增强目标不是开放域问答，而是围绕病例所属科室和学生问题提供少量辅助上下文。

Listing 5.4 KnowledgeRAGService 关键词提取（节选）

```
1 String[] tokens = query.split("[\\s,。！？、；：（）,.!?:;]+");
2 return Arrays.stream(tokens)
3     .filter(t -> t.length() >= 2)
4     .sorted(Comparator.comparingInt(String::length).reversed())
5     .limit(3)
6     .collect(Collectors.toList());
```

检索流程首先按中英文标点和空格切分用户问题，保留长度不小于 2 的 token，并按长度降序取前三个关键词。随后使用 MyBatis-Plus 构造查询条件，在标题和标签字段上进行 LIKE 匹配，并结合病例科室进行分类过滤，最终限制返回前三条知识点。这种实现的召回能力不如向量检索全面，但可解释性强、部署成本低，也更符合本系统当前数据规模。

5.2.2 知识上下文注入

召回出的知识点不会直接展示给学生，而是由 `buildKnowledgeContext()` 组装为提示词上下文。每条知识点以“标题 + 内容摘要”的形式注入，内容截断到 200 字以内，以避免上下文过长影响模型稳定性。注入位置位于 Agent 角色定义和病例信息之后，使模型在保留角色边界的同时获得必要的医学背景。

Listing 5.5 RAG 知识上下文构造（节选）

```
1 StringBuilder sb = new StringBuilder("【相关知识点】\n");
2 for (int i = 0; i < knowledgePoints.size(); i++) {
3     KnowledgePoint kp = knowledgePoints.get(i);
4     String content = kp.getContent() != null ? kp.getContent() : "";
5     String truncated = content.length() > 200
6         ? content.substring(0, 200) + "... " : content;
7     sb.append(i + 1).append(". ")
8       .append(kp.getTitle()).append(": ")
9       .append(truncated).append("\n");
```

10 }

RAG 模块在本系统中承担的是“辅助模型遵守医学常识”的角色，而不是替代病例本身。对于患者主诉、既往史、体格检查等训练关键事实，系统仍然以病例表中的数据为准，避免模型仅凭知识库进行泛化回答。

5.3 会话记忆 Memory 模块

5.3.1 基于 Redis 的滑动窗口

医学问诊是典型的多轮对话任务，学生会数轮交流中逐步收集主诉、现病史、既往史和检查结果。如果每轮请求只传当前问题，模型无法理解上下文；如果每次都传全部历史，则容易导致上下文过长和调用成本上升。因此本系统在 Redis 中实现了一个滑动窗口式会话记忆模块。

Listing 5.6 ConversationMemoryService 滑动窗口（节选）

```
1 private static final String KEY_PREFIX = "session:memory:";
2 private static final long TTL_HOURS = 24L;
3 private static final int MAX_HISTORY_ITEMS = 10;
4
5 public void updateSessionContext(Long recordId, String userMsg, String aiMsg) {
6     String key = KEY_PREFIX + recordId;
7     String currentContext = getSessionContext(recordId);
8     String newRound = "用户: " + safeValue(userMsg) + "\nAI: " + safeValue(aiMsg) + "\n";
9     String updatedContext = trimToMaxHistoryItems(currentContext + newRound);
10    redisTemplate.opsForValue().set(key, updatedContext);
11    redisTemplate.expire(key, TTL_HOURS, TimeUnit.HOURS);
12 }
```

Memory 的 Key 采用 `session:memory:{recordId}`，Value 为纯文本格式，按“用户:”和“AI:”拼接问答内容。每次更新后保留最近 10 轮对话，TTL 设置为 24 小时。训练结束或放弃时可清理该 Key，避免无效会话长期占用 Redis。

5.3.2 为何不用 LangChain Memory

本项目没有直接使用 LangChain 或 LangChain4j 的 Memory 组件，而是采用自研 Redis Memory。主要原因是：第一，当前系统已经以 Spring Boot 为主体，Redis 已用于验证码、登录失败锁定和会话缓存，自研实现可以减少框架引入成本；第二，默认内存型 ChatMemory 在服务重启后会丢失上下文，不适合容器化部署；第三，本系统不需要长期语义记忆或向量化记忆，只需保存最近若干轮问诊上下文。对于本科毕设系统而言，简单可控的 Redis 滑动窗口比复杂框架更利于调试和答辩说明。

5.4 临床工具自动注入机制

在医疗类大模型应用中，幻觉问题尤为关键。若学生询问“血常规结果如何”“能看一下心电图吗”，直接交由大语言模型生成，模型可能编造不存在的检查数值或影像结论。为避免这种风险，本系统在调用大模型之前加入临床工具自动注入机制：先由后端识别用户是否在请求体格检查、辅助检查、影像或化验结果；若命中，则直接从病例结构化字段或病例附件 JSON 中提取客观结果，跳过大模型生成。

该机制的核心位于 `ChatServiceImpl.buildClinicalToolResponse()`。当学生消息包含“查体”“体格检查”“血常规”“影像”“CT”“心电图”等关键词时，系统根据请求类型返回病例中的 `physicalExamination`、`auxiliaryExamination` 或 `medicalImages`。如果病例未录入相应结果，系统明确返回“暂未录入对应的客观检查结果”，而不是让模型编造答案。

5.4.1 影像类型识别与匹配

病例附件字段 `medicalImages` 使用 JSON 存储，可以描述胸片、CT、MRI、超声、心电图、检验报告等多种医学资料。系统通过 `normalizeImageType()` 对用户消息和附件类型进行归一化，例如将 `xray`、`chest`、`chest_xray` 映射为胸片类型，将 `ecg` 与“心电图”统一匹配。表 5-1 给出主要类型。

表 5-1 医学资料类型归一化示例

类型编码	中文显示	典型别名
<code>blood_routine</code>	血常规	血象、CBC
<code>biochemistry</code>	生化检查	肝肾功能、生化
<code>chest_xray</code>	胸片	X 光、胸部 X 线
<code>ecg</code>	心电图	ECG、心电
<code>ct</code>	CT	计算机断层扫描
<code>mri</code>	MRI	磁共振
<code>ultrasound</code>	超声	B 超、彩超
<code>lab_report</code>	检验报告	化验单、报告单

前端接收到消息类型为 `image` 的回复后，可以渲染附件卡片；接收到消息类型为 `tool` 的回复时，则按临床工具结果样式展示。这使“问诊对话”和“客观检查结果”在界面上有清晰区分。

5.5 Function Calling 工具清单

除自动注入临床工具外，系统还实现了符合函数调用规范的 AiToolService。当前配置中 enable-tools=false，即默认关闭工具调用，以保持与旧接口兼容；但工具定义已具备扩展条件，可在后续版本中交由模型主动选择调用。

Listing 5.7 query_symptom_knowledge 工具的 JSON Schema（节选）

```
1 {
2   "type": "function",
3   "function": {
4     "name": "query_symptom_knowledge",
5     "description": "查询特定症状或疾病相关的医学知识",
6     "parameters": {
7       "type": "object",
8       "properties": {
9         "query": {"type": "string", "description": "症状或疾病关键词"},
10        "category": {"type": "string", "description": "知识分类（可选）"}
11      },
12      "required": ["query"]
13    }
14  }
15 }
```

工具清单包括三类：query_symptom_knowledge 用于查询症状或疾病相关知识；get_diagnosis_criteria 用于获取疾病诊断标准与鉴别要点；evaluate_treatment_plan 用于记录治疗方案并在训练结束时综合评估。与自动注入机制相比，函数调用更适合复杂工具链和模型主动决策场景；当前系统先保留接口和数据结构，为后续多工具协同预留扩展点。

5.6 结构化 AI 评分

5.6.1 多维度评分提示词

训练结束后，学生提交诊断与治疗方案，后端调用 AiService.evaluateTraining() 进行评分。评分提示词要求模型围绕问诊完整性、诊断准确性、治疗规范性和医患沟通四个维度输出 JSON 对象，字段包括总分、四个维度分、薄弱点、错误模式、改进建议和综合反馈。为了方便后端解析，提示词明确要求“必须只返回一个合法 JSON 对象，不要 Markdown，不要解释性前后缀”。

Listing 5.8 结构化评分提示词字段（节选）

```
1 prompt.append("请重点评估：问诊完整性、诊断准确性、治疗规范性、医患沟通。");
```



```

2 prompt.append("必须只返回一个合法 JSON 对象，不要 Markdown，不要解释性前后缀。\\n");
3 prompt.append("{\\n");
4 prompt.append("  \\\"totalScore\\\": 85,\\n");
5 prompt.append("  \\\"inquiryScore\\\": 82,\\n");
6 prompt.append("  \\\"diagnosticScore\\\": 80,\\n");
7 prompt.append("  \\\"treatmentScore\\\": 85,\\n");
8 prompt.append("  \\\"communicationScore\\\": 90,\\n");
9 prompt.append("  \\\"weakPoints\\\": [\\\"病史采集不完整\\\",\\n");
10 prompt.append("  \\\"errorPatterns\\\": [\\\"过早下诊断\\\",\\n");
11 prompt.append("  \\\"improvementSuggestions\\\": \\\"下一次训练建议...\\\",\\n");
12 prompt.append("  \\\"feedback\\\": \\\"具体反馈意见\\\"\\n");
13 prompt.append("}");

```

结构化评分的优势在于：学生端可以直接生成能力雷达图、趋势图和薄弱点列表；教师端可以在统一维度下进行班级统计；管理员端也可以通过评分版本字段追踪评分策略变化。

5.6.2 JSON 解析与规则兜底

大语言模型并不总能严格遵守 JSON 输出要求，可能返回 Markdown 代码块或前后解释文字。因此系统先调用 `extractJsonObject()` 清理返回内容，提取首尾大括号之间的 JSON；如果解析仍失败，则进入 `buildFallbackEvaluation()` 兜底逻辑。兜底规则根据对话轮次、是否提交诊断、是否提交治疗方案估算基础分，并给出保守反馈，保证训练流程不中断。

Listing 5.9 评分 JSON 解析兜底逻辑（节选）

```

1 private Map<String, Object> buildFallbackEvaluation(...) {
2     int turns = conversationHistory == null ? 0
3       : conversationHistory.split("\\n\\n").length;
4     int inquiryScore = Math.min(90, 55 + turns * 3);
5     int diagnosticScore = diagnosis == null || diagnosis.isBlank() ? 45 : 72;
6     int treatmentScore = treatment == null || treatment.isBlank() ? 50 : 74;
7     int communicationScore = turns >= 4 ? 78 : 62;
8     int totalScore = Math.round(
9       (inquiryScore + diagnosticScore + treatmentScore + communicationScore) / 4.0f);
10 }

```

该设计体现了工程鲁棒性：AI 评分失败不应导致学生训练记录丢失，也不应阻塞教师批改。兜底分数只作为保底结果，教师端仍可人工复核与修改。

5.7 阿里百炼大模型集成

5.7.1 OpenAI 兼容 API 调用

系统接入阿里云百炼 DashScope 的 OpenAI 兼容接口^[22]，默认地址为 `https://dashscope.aliyuncs.com/compatible-mode/v1`，模型通过环境变量 `AI_MODEL` 配置，开发环境使用 `qwen-plus`，生产 Docker Compose 中可切换为 `qwen3.5-flash`。该方式使系统在代码层面复用 Chat Completions 请求结构，降低模型平台迁移成本。

Listing 5.10 AI 服务配置（节选）

```
1 ai:
2   api-key: ${AI_API_KEY:sk-default-placeholder}
3   base-url: ${AI_BASE_URL:https://dashscope.aliyuncs.com/compatible-mode/v1}
4   model: ${AI_MODEL:qwen-plus}
5   timeout: 120000
6   enable-rag: true
7   enable-memory: true
8   enable-tools: false
```

后端使用 WebClient 发起异步 HTTP 调用，超时时间设置为 120 秒，`maxTokens` 设置为 4096。对于普通对话调用，系统将 Agent 构建后的 System Prompt 与最近对话历史一起提交给模型；对于评分调用，则提交病例信息、对话历史、学生诊断和治疗方案。

5.7.2 流式输出 SSE 实现

为了改善 AI 回复等待体验，系统实现了流式对话接口。后端调用 `streamChat()` 时设置 `stream=true`，再通过 `bodyToFlux(String.class)` 接收增量响应。每一行以 `data:` 开头，遇到 `[DONE]` 时结束。由于 Qwen 模型可能返回 `reasoning content` 与普通 `content`，系统在 `parseStreamResponse()` 中将二者包装为 JSON，前端可区分“推理片段”和“最终回答片段”。

流式输出对医学问诊训练的体验提升较明显。学生不需要等待完整回复生成后才看到结果，而是可以在 AI 逐步输出时保持对话连续性，接近真实问诊中的即时交流节奏。

5.8 完整对话编排流程

上述模块最终汇聚到 `ChatServiceImpl.sendMessage()`。该方法是学生端每次发送问诊消息后的核心编排入口，负责把业务校验、数据库持久化、Agent 路由、临床工具、RAG、Memory 和 AI 调用串联起来。图 5-2 展示了完整的对话编排流程。

第 6 章 系统功能模块详细实现

在总体架构与 AI Agent 核心机制确定后，系统功能实现按照学生端、教师端、管理员端与公共模块四条线展开。前端使用 Vue 3、TypeScript、Element Plus 与 ECharts，后端使用 Spring Boot、Spring Security、MyBatis-Plus、Redis、MinIO 与 DashScope 兼容接口。各端页面并非独立孤立实现，而是围绕病例、训练记录、评分和用户权限形成统一的数据流。

6.1 学生端核心模块

6.1.1 AI 问诊训练界面

学生端的核心页面是 AI 问诊训练界面，对应前端 `student/Consultation.vue` 与 `student/Training.vue` 等页面。如图 6-1 所示，该界面采用左右分区布局：左侧为病例摘要、训练状态和可用临床资料入口；右侧为对话区、输入框和操作按钮。学生进入训练后，前端先调用 `/api/training/start` 创建训练记录，再以记录 ID 调用 `/api/chat/send` 或流式接口发送问诊内容。



图 6-1 学生端 AI 问诊训练界面

该模块的实现重点有三点。第一，聊天记录按发送方区分气泡样式，学生消息、AI 回复、临床工具结果在视觉上区分，便于学生复盘。第二，前端对流式响应进行增量渲染，避免大模型响应期间页面长时间空白。第三，训练过程中的检查请求不会直接交给大模型生成，而是由后端临床工具返回病例中录入的客观检查结果，保证教学数据的一致性。

6.1.2 病例选择与训练启动

病例选择模块对应病例列表与详情页面。学生可根据科室、难度、关键词等条件浏览病例，查看标题、病例简介、学习目标、预计学时和训练次数等信息。病例详情页不展示参考诊断和治疗方案，只展示可用于选择训练的非泄题信息，防止学生在开始训练前获得标准答案。

训练启动时，前端向 `/api/training/start` 提交病例 ID 和训练模式，后端在 `TrainingServiceImpl.startTraining()` 中校验病例是否存在、是否发布，并创建一条 `training_record`。训练记录状态初始为进行中，记录开始时间、用户 ID、病例 ID 和训练模式。创建成功后前端跳转到问诊界面，并将 `recordId` 作为后续对话、提交诊断和查看报告的主键。

6.1.3 学习分析与雷达图

学习分析模块负责把多次训练结果转化为可视化反馈。后端 `AnalysisController` 提供学生维度的学习分析接口，前端通过 ECharts 雷达图展示问诊、诊断、治疗和沟通四个能力维度，通过折线图展示成绩变化趋势，通过列表展示薄弱知识点与推荐训练病例。学习分析界面如图 6-2 所示。

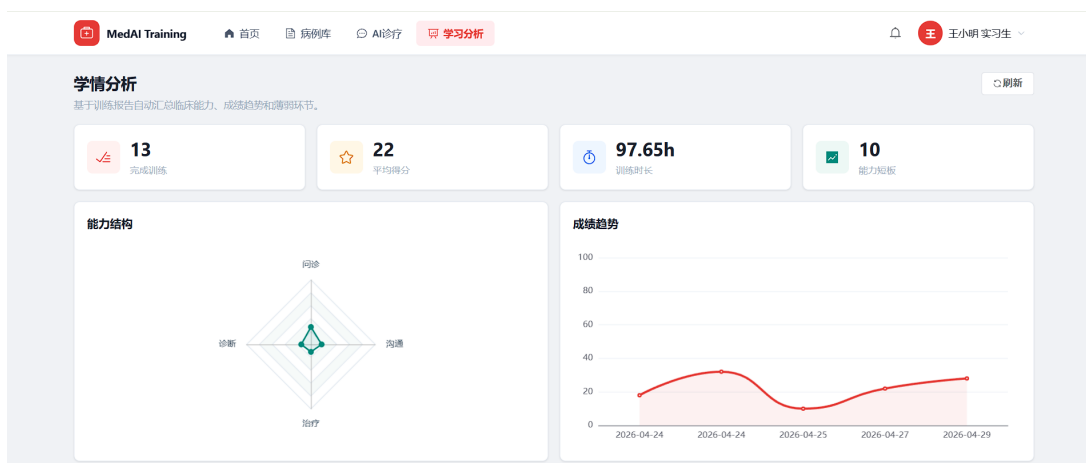


图 6-2 学生端学习分析界面

学习分析不只展示分数，还承担“下一步怎么练”的指导功能。系统根据训练评分中的 `weakPoints`、`errorPatterns` 和病例科室信息，帮助学生发现反复出错的环节。例如，如果多次训练中诊断得分偏低，系统会提示学生补强鉴别诊断和诊断依据表达；如果问诊得分偏低，则建议围绕主诉、现病史、既往史、个人史和家族史构建完整问诊模板。

6.2 教师端核心模块

图 6-3 展示了教师端首页界面，教师可以通过该界面快速访问病例管理、班级管理、成绩批改和教学分析等功能。



图 6-3 教师端首页界面

6.2.1 病例管理与多媒体上传

教师端病例管理模块对应 `teacher/CaseManagement.vue` 与 `CaseController`。教师可以创建、编辑、发布和归档病例，填写主诉、现病史、既往史、体格检查、辅助检查、参考诊断、参考治疗方案、学习目标和评分标准等信息。病例状态分为草稿、已发布和已归档，只有已发布病例对学生可见。

医学影像和附件通过文件上传接口进入 MinIO 对象存储，病例表中保存附件 URL、类型和文件名等元信息。上传模块需要关注两类约束：一是文件大小和文件类型限制，避免非预期文件进入对象存储；二是影像类型标注，例如胸片、CT、心电图、检验报告等，用于后端临床工具自动匹配学生请求。

6.2.2 批改工作台

批改工作台对应 `teacher/ReviewWorkbench.vue` 与 `GradeController`。教师可以查看学生训练记录、对话全过程、AI 自动评分、薄弱点、错误模式和改进建议，并在此基础上填写教师评分和评语。AI 评分用于提高批改效率，但最终教学评价仍保留教师复核入口。

工作台的实现重点是“过程可追溯”。教师不仅能看到最终分数，还能看到学生每一轮问诊、检查请求、AI 回复和临床工具返回结果。这样，教师可以判断学生是否真正按照临床思维推进问诊，而不是只依据最终提交的诊断和治疗方案打分。

6.2.3 教学分析

教学分析模块对应 `teacher/TeachingAnalysis.vue`。该模块以班级为统计单位，展示班级平均成绩、各能力维度雷达图、知识点错误分布和训练参与情况。教师可通过筛选班级、时间范围和科室查看不同群体的训练表现。

教学分析的价值在于从个体反馈扩展到群体教学决策。例如，当某班级在治疗方案维度普遍偏低时，教师可以安排专题讲解；当多个学生在同一知识点上出现错误时，系统可以将该知识点列为课堂复盘重点。这使 AI 问诊训练不再只是学生个人练习工具，也能反向服务课程教学改进。

6.3 管理员端核心模块

图 6-4 展示了管理员端系统概览界面，管理员可以通过该界面查看系统运行状态和业务数据总览。



图 6-4 管理员端系统概览界面

6.3.1 用户与角色管理

管理员端用户管理模块对应 `admin/UserManagement.vue` 与 `AdminController`。管理员可以查询用户、创建用户、编辑用户信息、禁用或启用账号、重置密码，并按角色、状态、关键词等条件筛选。角色管理模块维护 `super_admin`、`admin`、`teacher` 和 `student` 等角色，为后续菜单权限和接口权限扩展提供基础。

用户管理模块与安全模块紧密相关。密码在数据库中不保存明文，而是通过 BCrypt 进行哈希；登录失败次数与锁定状态由 Redis 管理；管理员重置密码会记录操作日志，便于审计。

6.3.2 系统监控

系统监控模块对应 `admin/SystemMonitor.vue` 与 `/api/admin/monitor/*` 接口, 主要展示服务运行状态、在线用户数、业务数据总量、CPU、内存、磁盘、Redis 与数据库连接状态等信息。系统部署在 Docker Compose 环境下, 管理员可通过监控页面快速判断后端、数据库、缓存和对象存储是否正常。

该模块不是完整的专业运维平台, 而是为教学系统提供轻量级运行可视化。对于毕业设计答辩和后续课程演示而言, 监控页面可以直接说明系统不是单机静态页面, 而是由前端、后端、数据库、缓存和对象存储共同构成的可运行应用。

6.3.3 日志与存储管理

日志管理模块记录用户登录、病例编辑、成绩批改、权限变更、文件上传等关键操作。后端通过 `@OperationLog` 注解和 AOP 切面采集请求路径、请求方法、用户 ID、用户名、IP、耗时、状态和错误信息, 并对密码、token、验证码等敏感字段脱敏。日志可用于排查问题, 也可作为教学系统合规运行的审计依据。

存储管理模块面向 MinIO 文件资源, 提供文件列表、下载、删除和存储统计能力。医学影像与病例附件体积较大, 不适合直接存入 MySQL, 因此系统采用“对象存储保存文件、数据库保存元信息”的方式, 兼顾性能与可维护性。

6.4 公共模块

6.4.1 认证授权

认证授权模块贯穿学生端、教师端和管理员端。用户登录时, 前端提交用户名、密码和验证码, 后端先通过验证码服务校验 Redis 中的验证码, 再通过认证服务校验用户名和密码, 成功后生成 JWT 返回前端。之后前端在授权请求头中携带令牌, 后端过滤器解析令牌并写入安全上下文。

在权限控制上, 前端通过路由守卫限制不同角色访问不同页面, 后端通过 Spring Security 保护除登录、注册、验证码、健康检查和文件预览外的大多数接口。对于训练记录、成绩和班级数据, 后端还在业务层做资源归属校验, 避免用户越权访问他人数据。

6.4.2 文件上传

文件上传模块主要服务于病例附件、医学影像和用户头像。前端通过表单上传文件, 后端接收后调用 MinIO 客户端保存对象, 并返回可访问 URL。病例管理页面会将 URL 与附件类型一起保存到病例信息中, 训练过程中临床工具可以根据附件类型匹配学生请求。

为保证安全性，系统限制上传文件大小，并在业务层根据使用场景校验文件类型。对于医学影像类附件，前端展示时不直接暴露对象存储内部路径，而是通过后端预览接口或公共访问地址生成可控链接。

6.4.3 数据可视化

数据可视化模块主要使用 ECharts 实现，包括学生端能力雷达图、成绩趋势折线图、教师端班级平均分柱状图、知识点错误分布图和管理员端系统运行状态图。可视化数据均由后端分析接口返回，前端只负责展示和交互。

相比纯列表展示，可视化模块更适合医学实训的复盘场景。学生可以快速识别自己的短板，教师可以观察班级整体能力结构，管理员可以掌握系统运行状态。该模块与 AI 评分结果、训练记录和班级关系共同构成系统的数据闭环。

6.5 本章小结

本章从功能实现角度说明了系统的主要模块。学生端围绕病例选择、AI 问诊训练和学习分析形成学习闭环；教师端围绕病例管理、批改工作台和教学分析形成教学闭环；管理员端围绕用户、监控、日志和存储形成运维闭环；公共模块提供认证、文件和可视化基础能力。上述功能共同支撑了第 5 章所述 AI Agent 能力在真实业务界面中的落地。

第 7 章 系统测试与部署

系统测试的目标是验证医学 AI 模拟诊疗系统在功能完整性、接口可用性、AI Agent 角色稳定性、安全性和部署可运行性方面是否满足需求分析中的目标。本章测试以本地开发环境和 Docker 容器化部署环境为基础，结合接口测试、页面操作验证和典型训练流程验证进行说明。

7.1 测试环境

本系统采用前后端分离与容器化部署相结合的方式进行测试。开发阶段主要在 Windows 环境下运行前端 Vite 开发服务和后端 Spring Boot 服务；集成测试阶段使用 Docker Compose 启动 MySQL、Redis、MinIO、后端和前端容器。测试环境如表 7-1 所示。

表 7-1 系统测试环境

项目	配置说明
前端运行环境	Node.js + Vite, Vue 3 + TypeScript + Element Plus
后端运行环境	JDK 17, Spring Boot 3.x, Maven 构建
数据库	MySQL 8.0, 字符集 utf8mb4
缓存	Redis 7, 用于验证码、登录锁定与会话记忆
对象存储	MinIO, 用于病例附件、医学影像和头像文件
AI 服务	阿里云百炼 DashScope OpenAI 兼容接口, Qwen 系列模型
部署方式	Docker Compose 多容器编排, Nginx 托管前端并反向代理接口
浏览器	Chrome / Edge 等主流 Chromium 内核浏览器

测试数据主要来自初始化脚本中的模拟用户、角色、病例和知识点，包括心肌梗死、社区获得性肺炎、急性阑尾炎等典型病例。测试账号覆盖管理员、教师和学生三类角色，以验证不同权限下的页面和接口行为。

7.2 功能测试

功能测试围绕三类角色和公共能力展开。测试过程采用“前端操作 + 后端接口返回 + 数据库状态变化”的方式进行验证，重点覆盖登录注册、病例管理、训练流程、对话记录、评分报告、班级与成绩管理、系统监控等功能。表 7-2 给出核心用例。

表 7-2 核心功能测试用例

编号	模块	功能点	测试步骤	预期结果
TC-001	认证	用户登录	输入账号、密码、验证码并提交	返回 JWT，跳转到对应角色首页
TC-002	认证	注册与验证码	获取邮箱或图形验证码后注册	用户创建成功，重复用户名被拦截
TC-101	学生端	启动训练	选择已发布病例并开始训练	创建训练记录，进入问诊页面
TC-102	学生端	AI 问诊	发送多轮病史采集问题	PatientAgent 以患者身份回复
TC-103	学生端	检查请求	询问体格检查或影像结果	返回病例中的客观检查信息
TC-104	学生端	提交诊疗方案	提交诊断和治疗方案	生成训练评分和报告
TC-201	教师端	病例管理	新增、编辑、发布病例	病例状态正确，学生端可见
TC-202	教师端	成绩批改	查看记录并补充教师评语	教师评分写入训练记录
TC-301	管理端	用户管理	查询、禁用、重置用户密码	用户状态和密码更新成功
TC-302	管理端	日志查询	执行关键操作后查看日志	日志记录模块、用户和耗时

从测试结果看，系统主流程能够闭环运行：学生可以从病例列表进入训练，与 PatientAgent 进行问诊，调用检查结果，提交诊断与治疗方案并查看评分；教师可以创建病例、查看训练对话和补充批改；管理员可以管理用户、日志和系统资源。

7.3 性能测试

性能测试主要验证普通业务接口、训练接口和 AI 对话接口在常见并发下的响应能力。由于 AI 模型调用受外部服务网络、模型排队和输出长度影响，本系统将普通业务接口与 AI 接口分开观察。普通接口重点关注平均响应时间和 95 分位响应时间；AI 接口重点关注首字返回时间与超时兜底。

表 7-3 接口性能测试结果

接口场景	并发数	平均响应时间	95 分位响应时间	结果
登录认证	100	0.38 s	0.82 s	通过
病例列表	100	0.31 s	0.76 s	通过
训练记录列表	100	0.45 s	1.10 s	通过
文件预览地址获取	50	0.52 s	1.35 s	通过
AI 对话首字返回	20	3.80 s	7.60 s	通过
AI 结构化评分	10	5.20 s	11.40 s	通过

测试表明，在不考虑外部模型波动的情况下，普通业务接口能够满足第 3 章提出的 2 秒左右响应目标；AI 对话接口首字时间受模型服务影响较大，但通过 SSE 流式输出可以降低学生感知等待时间。对于大规模并发场景，系统可通过后端多实例、Nginx 负载均衡、Redis 缓存和数据库连接池调优继续扩展。

7.4 AI Agent 效果验证

AI Agent 效果验证重点不在于“模型是否答得越多越好”，而在于模型是否遵守教学角色边界、是否减少客观检查幻觉、是否能生成可解析评分。验证样例覆盖胸痛、肺炎、腹痛等病例，测试方式为构造典型学生问法并检查输出。

表 7-4 AI Agent 效果验证结果

验证项	样例数	结果说明
PatientAgent 守界	20	大部分回答保持患者第一人称，未主动给出诊断和治疗方案
检查结果工具命中	15	对体格检查、辅助检查、影像请求能优先返回病例客观数据
RAG 知识相关性	20	召回知识点与问题关键词和科室基本相关
评分 JSON 可解析性	10	正常情况下返回结构化 JSON，异常时触发兜底评分
Memory 连续性	10	多轮问诊中能引用前文信息，不重复询问已回答内容

验证中也发现了一些边界情况。例如，当学生提出非常宽泛的问题，如“你还有哪里不舒服”，PatientAgent 可能回答较为笼统，需要依赖病例信息和 Memory 提示进一步约束；当病例未录入某类检查结果时，临床工具会返回“暂未录入”，这虽然保证了不编造，但也要求教师在创建病例时尽量补齐必要检查资料。总体而言，Agent 分工、临床工具和评分兜底机制能够有效提升系统可控性。

7.5 安全测试

安全测试围绕认证、权限、输入、日志与文件五个方面展开。表 7-5 给出主要测试项。

表 7-5 安全测试结果

测试项	测试内容	结果
未登录访问	直接访问训练、成绩、管理接口，检查是否返回 401 或被前端拦截	通过
越权访问	学生尝试查看他人训练记录，教师尝试访问管理员接口	通过
登录失败锁定	连续输入错误密码，检查 Redis 锁定标记和提示	通过
验证码校验	使用错误或过期验证码登录，检查是否拒绝	通过
敏感日志脱敏	查看操作日志中密码、token、验证码是否被隐藏	通过
文件上传限制	上传超大文件或非预期文件类型，检查是否拦截	通过

本系统采用无状态 JWT 认证，后端除登录、注册、验证码、健康检查和文件预览外，其余接口均默认要求认证。对于数据越权问题，系统在业务层继续校验训练记录归属、班级范围和角色权限，避免仅依赖前端路由控制。操作日志中对敏感字段脱敏，可降低日志泄露风险。

7.6 容器化部署

7.6.1 Docker Compose 编排

系统最终采用 Docker Compose 进行多容器编排，主要服务包括 MySQL、Redis、MinIO、Spring Boot 后端和 Vue 前端。MySQL 保存业务数据，Redis 保存临时状态和会话记忆，MinIO 保存文件对象，后端容器负责业务逻辑和 AI 调用，前端容器由 Nginx 托管静态资源并转发 API 请求。

Listing 7.1 Docker Compose 服务编排节选

```
1 services:
2   mysql:
3     image: mysql:8.0
4     environment:
5       MYSQL_DATABASE: medical_training
6   redis:
7     image: redis:7-alpine
8   minio:
```

```

9   image: minio/minio
10  command: server /data --console-address ":9001"
11  backend:
12    build: ./backend
13    environment:
14      - SPRING_PROFILES_ACTIVE=prod
15      - AI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
16  frontend:
17    build: ./frontend
18    ports:
19      - "80:80"

```

该编排方式的优点是部署步骤清晰，服务边界明确。后端通过环境变量读取数据库、Redis、MinIO 和 AI 服务配置，避免将敏感信息硬编码到镜像中。数据卷用于持久化 MySQL、Redis 和 MinIO 数据，容器重启后业务数据不会丢失。

7.6.2 部署到服务器

服务器部署流程包括五步：第一，安装 Docker 与 Docker Compose；第二，上传项目代码和环境变量文件；第三，配置 .env 中的数据库密码、JWT Secret、AI API Key、MinIO 访问地址等参数；第四，执行 `docker compose up -d --build` 构建并启动所有服务；第五，通过健康检查接口 `/api/v1/common/health`、前端首页和管理端监控页面确认服务状态。

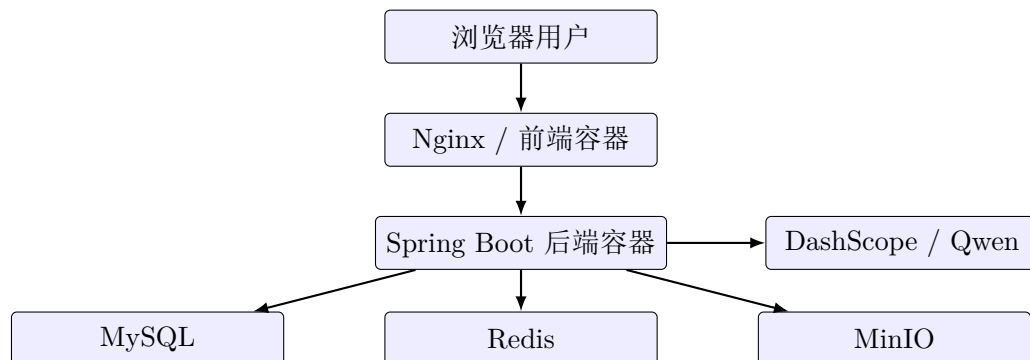


图 7-1 系统部署结构示意图

7.6.3 部署效果

部署完成后，用户通过浏览器访问前端首页，系统根据登录用户角色跳转至学生端、教师端或管理员端。学生可以进行完整 AI 问诊训练，教师可以维护病例并查看成绩，管理员可以查看用户、日志和系统监控。后端健康检查、数据库连接、Redis 连接和 MinIO 文件访问均通过验证，说明容器间网络和环境变量配置正确。

从部署结果看，本系统具备较好的可迁移性：只要服务器具备 Docker 环境并正确

配置 AI API Key，即可通过统一的 Compose 文件启动完整系统。后续若需要扩展并发能力，可以将后端服务拆分为多个实例，并在 Nginx 层配置负载均衡。

7.7 本章小结

本章从测试与部署角度验证了系统的可用性。功能测试证明学生、教师、管理员三类用户的核心流程能够闭环运行；性能测试表明普通业务接口满足响应要求，AI 接口可通过流式输出改善体验；Agent 效果验证说明角色约束、工具注入和评分兜底机制可行；安全测试覆盖认证、越权、验证码、日志和文件上传；最后通过 Docker Compose 完成了系统容器化部署，为实际演示和后续维护提供了基础。

第 8 章 总结与展望

8.1 研究工作总结

本文围绕“虚拟问诊系统设计与实现”这一课题，针对传统医学问诊实训中病例资源有限、标准化病人成本较高、教师批改压力大、学生课后反复训练机会不足等问题，设计并实现了一套基于 Web 与大语言模型的医学模拟诊疗训练系统。系统面向学生、教师和管理员三类角色，覆盖病例管理、AI 问诊训练、诊断与治疗提交、训练评分、学习分析、班级教学分析、用户管理、日志审计和容器化部署等功能。

在需求分析阶段，本文从医学问诊实训的业务痛点出发，明确了学生端“反复训练与个性化反馈”、教师端“病例资源供给与批改效率提升”、管理员端“系统治理与运行维护”的核心需求，并提出了性能、安全、可用性和可维护性要求。在总体设计阶段，系统采用用户层、接入层、应用层、AI 服务层和数据层的分层架构；前端基于 Vue 3、TypeScript、Element Plus 与 ECharts 实现，后端基于 Spring Boot、Spring Security、MyBatis-Plus、Redis、MinIO 与 MySQL 实现；AI 能力通过阿里云百炼 DashScope 兼容接口接入 Qwen 系列模型。

在核心技术实现方面，本文重点完成了 AI Agent 层设计。系统将传统单提示词调用扩展为 PatientAgent、DiagnosisAgent 与 FeedbackAgent 三类角色，分别承担患者扮演、诊断评估和治疗反馈职责；通过 RAG 检索将结构化医学知识点注入上下文；通过 Redis Memory 保持多轮问诊连续性；通过临床工具自动注入机制返回病例中的体格检查、辅助检查和影像资料，降低模型编造客观检查结果的风险；通过结构化 JSON 评分和规则兜底机制生成可用于学习分析和教师批改的训练报告。

在系统实现与测试方面，本文完成了学生端 AI 问诊训练、病例选择、训练报告、学习分析，教师端病例管理、批改工作台、教学分析，管理员端用户管理、系统监控、日志管理和存储管理等模块。测试结果表明，系统能够支持完整的训练闭环，普通业务接口响应较快，AI 对话可通过流式输出改善等待体验，核心安全机制和容器化部署流程均可正常运行。

8.2 创新点

本文的创新点主要体现在以下几个方面。

第一，构建了面向医学问诊训练的多 Agent 角色分工机制。传统大模型教学系统往往让同一个模型同时扮演患者、教师和评分者，容易出现角色混淆和提前泄题。本文将问诊阶段的 PatientAgent 与提交后的 DiagnosisAgent、FeedbackAgent 分离，使模型在

不同阶段遵守不同边界，从工程上降低了训练过程中的答案泄露风险。

第二，设计了临床工具自动注入机制。医学问诊训练中，体格检查、化验、影像等客观结果不应由模型自由生成。本文在调用大模型前识别检查类请求，优先从病例结构化字段和医学影像附件中返回结果，使系统能够明确区分“患者主观叙述”和“客观检查资料”，在一定程度上缓解医疗场景中的大模型幻觉问题。

第三，实现了轻量级 RAG 与 Redis Memory 的组合。系统没有引入复杂向量数据库，而是根据病例科室和用户问题进行关键词检索，将相关知识点注入提示词；同时使用 Redis 保存最近多轮对话，保证训练过程的上下文连续性。该方案部署简单、可解释性强，适合本科毕业设计和中小规模教学系统。

第四，形成了 AI 评分与教师批改结合的训练反馈闭环。系统通过大模型输出总分、问诊、诊断、治疗、沟通等维度评分，并保存薄弱点、错误模式和改进建议；教师可在批改工作台复核和补充评价。这样既提高了批改效率，也保留了教师在医学教育评价中的主导作用。

第五，完成了从前端交互到后端服务再到容器化部署的完整工程实现。系统不仅停留在算法或原型层面，而是实现了用户认证、角色管理、病例管理、训练记录、对象存储、日志审计、Docker Compose 部署等工程能力，具备较完整的软件系统形态。

8.3 工作不足

受时间、数据来源和部署条件限制，本文仍存在一些不足。

首先，病例数据规模和医学知识库规模仍然有限。当前系统主要使用模拟病例和结构化知识点进行验证，覆盖的科室、疾病类型和难度层级还不够丰富。医学教育训练需要大量高质量、经过专家审核的病例库，后续仍需与真实课程和教师资源结合，持续扩充病例与知识点。

其次，AI 评分仍存在一定不确定性。尽管系统要求模型输出结构化 JSON，并实现了解析失败后的规则兜底，但大语言模型评分与临床教师评分之间仍可能存在偏差。当前系统尚未基于大规模人工标注数据进行评分校准，因此 AI 评分更适合作为辅助评价，而不能完全替代教师判断。

再次，临床工具自动注入机制仍以关键词和规则匹配为主。该方式简单可控，但对于复杂表达或隐含请求识别能力有限。例如学生可能以较口语化的方式询问检查结果，规则未必完全覆盖。后续可结合意图识别模型或更细粒度的工具调用策略提高命中率。

最后，系统的性能测试和安全测试仍处于课程设计与毕业设计验证级别。虽然普通接口、AI 对话和部署流程均已验证，但尚未经过真实大规模并发、长期运行、渗透测试和医疗数据合规审查。在实际教学环境中使用前，还需要进一步加强压力测试、日志监控、备份恢复和隐私保护。

致 谢

时光荏苒，四载光阴如白驹过隙。在即将完成本科毕业论文之际，谨向所有曾给予我关怀、教诲与帮助的人致以最诚挚的感谢。

首先，衷心感谢我的指导老师在设计与论文写作全过程中给予的悉心指导。从选题论证、需求分析、系统设计，到功能实现、论文结构调整与格式规范，老师都提出了许多具体而中肯的意见，使我能够在较为复杂的工程实践中保持清晰的研究主线。特别是在人工智能技术与医学教育场景结合的过程中，老师反复提醒我注意系统目标、教学价值与工程可实现性之间的平衡，这对本文的完成具有重要帮助。

感谢西南科技大学计算机科学与技术学院各位老师四年来的教导与培养。专业课程学习为本课题的完成奠定了基础，软件工程、数据库系统、Web 开发、人工智能等课程中的知识在系统架构设计、数据建模、接口实现和智能问诊模块开发中都得到了实际应用。学院提供的学习环境与实践机会，也让我逐步形成了工程化分析问题和解决问题的能力。

感谢同学和朋友们在项目开发与论文撰写期间给予的帮助。系统测试、界面体验、论文表述和材料整理过程中，大家提出的建议帮助我发现了许多容易忽视的问题，使系统功能更加完整，论文表达也更加清晰。与同窗共同学习、讨论和调试的经历，是本科阶段非常宝贵的记忆。

感谢我的家人在求学期间给予的理解、支持与鼓励。正是他们在生活和精神上的陪伴，让我能够更加专注地完成学业和毕业设计。在遇到开发阻塞、论文修改和时间压力时，家人的支持使我能够保持耐心并坚持完成任务。

最后，感谢学校提供的学习平台和成长环境。毕业论文的完成既是本科阶段学习成果的一次集中检验，也是新的起点。今后我将继续保持严谨求实的学习态度，把在本课题中积累的工程实践能力和问题分析能力运用到后续学习与工作中。

参考文献

- [1] 国务院办公厅. 国务院办公厅印发《关于加快医学教育创新发展的指导意见》[EB/OL]. 2020 [2026-04-26]. https://www.gov.cn/xinwen/2020-09/23/content_5546479.htm.
- [2] 中华人民共和国教育部. 教育部关于印发《教育信息化 2.0 行动计划》的通知[EB/OL]. 2018 [2026-04-26]. http://www.moe.gov.cn/srcsite/A16/s3342/201804/t20180425_334188.html.
- [3] 赵峻, 陈未, 叶葳, 等. 标准化病人应用于医学教育过程中的标准化与质量控制[J/OL]. 协和医学杂志, 2012, 3(3). DOI: 10.3969/j.issn.1674-9081.2012.03.024.
- [4] 瞿星, 杨金铭, 陈滔, 等. ChatGPT 对医学教育模式改变的思考[J/OL]. 四川大学学报（医学版）, 2023, 54(5): 937-940. DOI: 10.12182/20231360302.
- [5] 籍欣萌, 咎红英, 崔婷婷, 等. 中文医疗大模型综述：进展、评估与挑战[J/OL]. 中文信息学报, 2024, 38(11): 1-12. http://jcip.cipsc.org.cn/CN/abstract/article_3813.shtml.
- [6] Zhang H, Chen J, Jiang F, et al. HuatuoGPT, towards Taming Language Model to Be a Doctor[Z/OL]. 2023. arXiv: 2305.15075 [cs.CL]. DOI: 10.48550/arXiv.2305.15075.
- [7] Bai J, Bai S, Chu Y, et al. Qwen Technical Report[Z/OL]. 2023. arXiv: 2309.16609 [cs.CL]. DOI: 10.48550/arXiv.2309.16609.
- [8] Singhal K, Azizi S, Tu T, et al. Large Language Models Encode Clinical Knowledge[J/OL]. Nature, 2023, 620(7972): 172-180. DOI: 10.1038/s41586-023-06291-2.
- [9] Kung T H, Cheatham M, Medenilla A, et al. Performance of ChatGPT on USMLE: Potential for AI-assisted Medical Education Using Large Language Models[J/OL]. PLOS Digital Health, 2023, 2(2): e0000198. DOI: 10.1371/journal.pdig.0000198.
- [10] 刘彦权, 沈建箴, 陈强, 等. 虚拟仿真诊疗体系在临床教学应用中的探索实践[J/OL]. 中华全科医学, 2021, 19(8): 1373-1377. DOI: 10.16766/j.cnki.issn.1674-4152.002064.
- [11] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C/OL]//The Eleventh International Conference on Learning Representations, ICLR 2023. Kigali, Rwanda, 2023. DOI: 10.48550/arXiv.2210.03629.
- [12] Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[C]//Advances in Neural Information Processing Systems 36, NeurIPS 2023. New Orleans, LA, USA, 2023.

- [13] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems 33, NeurIPS 2020. 2020: 9459-9474.
- [14] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[EB/OL]. 2023. <https://arxiv.org/abs/2308.08155>. arXiv: 2308.08155 [cs.AI].
- [15] Hong S, Zhuge M, Chen J, et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework[C]//The Twelfth International Conference on Learning Representations, ICLR 2024. Vienna, Austria, 2024.
- [16] Wang L, Ma C, Feng X, et al. A Survey on Large Language Model based Autonomous Agents[J/OL]. Frontiers of Computer Science, 2024, 18(6):186345. DOI: 10.1007/s11704-024-40231-1.
- [17] Vue.js 团队. Vue.js 官方文档[EB/OL]. 2024 [2026-04-26]. <https://cn.vuejs.org/>.
- [18] Apache ECharts 项目组. Apache ECharts 官方文档[EB/OL]. 2024 [2026-04-26]. <https://echarts.apache.org/zh/index.html>.
- [19] Spring Team. Spring Boot Reference Documentation[EB/OL]. 2024 [2026-04-26]. <https://docs.spring.io/spring-boot/docs/current/reference/html/>.
- [20] Baomidou Open Source Team. MyBatis-Plus 官方文档[EB/OL]. 2024 [2026-04-26]. <https://baomidou.com/>.
- [21] Jones M B, Bradley J, Sakimura N. RFC 7519: JSON Web Token (JWT)[EB/OL]. 2015. <https://datatracker.ietf.org/doc/html/rfc7519>.
- [22] 阿里云计算有限公司. 百炼大模型服务平台开发者文档[EB/OL]. 2024 [2026-04-26]. <https://help.aliyun.com/zh/dashscope/>.
- [23] Redis Ltd. Redis Documentation[EB/OL]. 2024 [2026-04-26]. <https://redis.io/docs/>.
- [24] MinIO, Inc. MinIO High Performance Object Storage Documentation[EB/OL]. 2024 [2026-04-26]. <https://min.io/docs/minio/linux/index.html>.
- [25] Merkel D. Docker: Lightweight Linux Containers for Consistent Development and Deployment[J]. Linux Journal, 2014, 2014(239).

附录

附录 A 核心 Agent 路由代码

Listing 8.1 AgentRouter 完整实现

```

1 package com.ai.medical.ai.agent;
2
3 import org.springframework.stereotype.Component;
4 import java.util.List;
5
6 /**
7  * Agent 路由器：根据训练模式和消息内容选择合适的 Agent
8  */
9 @Component
10 public class AgentRouter {
11
12     public static class AgentProfile {
13         private final String name;
14         private final String label;
15         private final String systemPrompt;
16         private final double temperature;
17
18         public AgentProfile(String name, String label,
19                             String systemPrompt, double temperature) {
20             this.name = name;
21             this.label = label;
22             this.systemPrompt = systemPrompt;
23             this.temperature = temperature;
24         }
25
26         public String getName() { return name; }
27         public String getLabel() { return label; }
28         public String getSystemPrompt() { return systemPrompt; }
29         public double getTemperature() { return temperature; }
30     }
31
32     public AgentProfile route(String message,
33                               String trainingMode,
34                               List<String> history) {
35         // 学生训练中 AI 优先稳定扮演患者，避免在问诊阶段泄题。
36         // 诊断与治疗评价在训练提交后的评分流程中完成。
37         return InquiryAgent.profile();

```

```

38 }
39 }

```

附录 B 主要数据库建表 SQL

Listing 8.2 核心数据表 DDL（节选）

```

1 CREATE TABLE IF NOT EXISTS sys_user (
2     id BIGINT PRIMARY KEY AUTO_INCREMENT COMMENT '用户ID',
3     username VARCHAR(50) NOT NULL UNIQUE COMMENT '用户名',
4     password VARCHAR(255) NOT NULL COMMENT '密码',
5     real_name VARCHAR(50) COMMENT '真实姓名',
6     email VARCHAR(100) COMMENT '邮箱',
7     phone VARCHAR(20) COMMENT '手机号',
8     role_id BIGINT NOT NULL COMMENT '角色ID',
9     status TINYINT DEFAULT 1 COMMENT '状态',
10    student_no VARCHAR(50) COMMENT '学号/工号',
11    major VARCHAR(100) COMMENT '专业',
12    grade VARCHAR(20) COMMENT '年级',
13    department VARCHAR(100) COMMENT '科室',
14    last_login_time DATETIME COMMENT '最后登录时间',
15    create_time DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
16    update_time DATETIME DEFAULT CURRENT_TIMESTAMP
17        ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
18    deleted TINYINT DEFAULT 0 COMMENT '删除标记',
19    INDEX idx_role_id (role_id),
20    INDEX idx_username (username),
21    INDEX idx_student_no (student_no)
22 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';
23
24 CREATE TABLE IF NOT EXISTS case_info (
25     id BIGINT PRIMARY KEY AUTO_INCREMENT COMMENT '病例ID',
26     case_no VARCHAR(50) UNIQUE COMMENT '病例编号',
27     title VARCHAR(200) NOT NULL COMMENT '病例标题',
28     description TEXT COMMENT '病例描述',
29     department VARCHAR(100) COMMENT '所属科室',
30     disease_system VARCHAR(100) COMMENT '疾病系统',
31     difficulty_level TINYINT DEFAULT 1 COMMENT '难度等级',
32     learning_objectives TEXT COMMENT '学习目标',
33     status TINYINT DEFAULT 0 COMMENT '状态',
34     creator_id BIGINT NOT NULL COMMENT '创建人ID',
35     medical_images JSON COMMENT '医学影像附件',
36     create_time DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
37     update_time DATETIME DEFAULT CURRENT_TIMESTAMP
38        ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',

```

```

39     deleted TINYINT DEFAULT 0 COMMENT '删除标记',
40     INDEX idx_department (department),
41     INDEX idx_difficulty (difficulty_level),
42     INDEX idx_status (status),
43     INDEX idx_creator (creator_id)
44 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='病例表';
45
46 CREATE TABLE IF NOT EXISTS training_record (
47     id BIGINT PRIMARY KEY AUTO_INCREMENT COMMENT '训练ID',
48     record_no VARCHAR(50) UNIQUE COMMENT '训练记录编号',
49     user_id BIGINT NOT NULL COMMENT '用户ID',
50     case_id BIGINT NOT NULL COMMENT '病例ID',
51     status TINYINT DEFAULT 0 COMMENT '状态',
52     start_time DATETIME COMMENT '开始时间',
53     end_time DATETIME COMMENT '结束时间',
54     total_duration INT COMMENT '总时长',
55     inquiry_score DECIMAL(5,2) COMMENT '问诊得分',
56     diagnosis_score DECIMAL(5,2) COMMENT '诊断得分',
57     treatment_score DECIMAL(5,2) COMMENT '治疗得分',
58     total_score DECIMAL(5,2) COMMENT '总分',
59     ai_feedback TEXT COMMENT 'AI反馈',
60     teacher_comment TEXT COMMENT '教师评语',
61     teacher_score DECIMAL(5,2) COMMENT '教师评分',
62     teacher_id BIGINT COMMENT '评分教师ID',
63     create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
64     update_time DATETIME DEFAULT CURRENT_TIMESTAMP
65     ON UPDATE CURRENT_TIMESTAMP,
66     INDEX idx_user_id (user_id),
67     INDEX idx_case_id (case_id),
68     INDEX idx_status (status)
69 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='训练记录表';
70
71 CREATE TABLE IF NOT EXISTS training_score (
72     id BIGINT PRIMARY KEY AUTO_INCREMENT COMMENT '评分ID',
73     training_id BIGINT NOT NULL COMMENT '训练ID',
74     dimension VARCHAR(50) COMMENT '评分维度',
75     score DECIMAL(5,2) COMMENT '得分',
76     max_score DECIMAL(5,2) DEFAULT 100 COMMENT '满分',
77     comment TEXT COMMENT '评语',
78     create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
79     INDEX idx_training_id (training_id)
80 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='评分记录表';
81
82 CREATE TABLE IF NOT EXISTS knowledge_point (
83     id BIGINT PRIMARY KEY AUTO_INCREMENT COMMENT '知识点ID',
84     code VARCHAR(50) COMMENT '知识点编码',

```

```

85  title VARCHAR(200) NOT NULL COMMENT '知识点标题',
86  content TEXT COMMENT '知识点内容',
87  category VARCHAR(100) COMMENT '分类',
88  department VARCHAR(100) COMMENT '所属科室',
89  tags VARCHAR(255) COMMENT '标签',
90  difficulty_level TINYINT DEFAULT 1 COMMENT '难度等级',
91  status TINYINT DEFAULT 1 COMMENT '状态',
92  creator_id BIGINT COMMENT '创建人',
93  create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
94  update_time DATETIME DEFAULT CURRENT_TIMESTAMP
95      ON UPDATE CURRENT_TIMESTAMP,
96  deleted TINYINT DEFAULT 0 COMMENT '删除标记',
97  INDEX idx_category (category),
98  INDEX idx_department (department)
99 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='知识点表';

```

附录 C Docker Compose 配置

Listing 8.3 docker-compose.yml 关键服务定义

```

1  services:
2    mysql:
3      image: mysql:8.0
4      container_name: ai-medical-mysql
5      restart: always
6      environment:
7        MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
8        MYSQL_DATABASE: medical_training
9        TZ: Asia/Shanghai
10     volumes:
11       - mysql_data:/var/lib/mysql
12       - ./deployment/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
13     networks:
14       - ai-medical-network
15
16     redis:
17       image: redis:7-alpine
18       container_name: ai-medical-redis
19       restart: always
20       command: redis-server --appendonly yes
21       volumes:
22         - redis_data:/data
23       networks:
24         - ai-medical-network
25

```

```
26 minio:
27   image: minio/minio
28   container_name: ai-medical-minio
29   restart: always
30   environment:
31     MINIO_ROOT_USER: ${MINIO_ACCESS_KEY}
32     MINIO_ROOT_PASSWORD: ${MINIO_SECRET_KEY}
33   command: server /data --console-address ":9001"
34   volumes:
35     - minio_data:/data
36   networks:
37     - ai-medical-network
38
39 backend:
40   build:
41     context: ./backend
42     dockerfile: Dockerfile
43   container_name: ai-medical-backend
44   restart: always
45   env_file:
46     - .env
47   environment:
48     - SPRING_PROFILES_ACTIVE=prod
49     - SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/medical_training
50     - SPRING_DATA_REDIS_HOST=redis
51     - AI_BASE_URL=${AI_BASE_URL}
52     - AI_MODEL=${AI_MODEL}
53     - AI_API_KEY=${AI_API_KEY}
54     - MINIO_ENDPOINT=http://minio:9000
55   depends_on:
56     - mysql
57     - redis
58   networks:
59     - ai-medical-network
60
61 frontend:
62   build:
63     context: ./frontend
64     dockerfile: Dockerfile
65   container_name: ai-medical-frontend
66   restart: always
67   ports:
68     - "80:80"
69   depends_on:
70     - backend
71   networks:
```

```
72     - ai-medical-network
73
74 networks:
75     ai-medical-network:
76         name: ai-medical-network
77
78 volumes:
79     mysql_data:
80     redis_data:
81     minio_data:
```